C# and Java: Comparing Programming Languages

Windows CE .NET

Kirk Radeck, Windows Embedded MVP
Senior Software Engineering Consultant

October 2003

Applies to:
- Microsoft® Windows® CE .NET
- Microsoft Windows XP Embedded
- Microsoft Pocket PC
- Microsoft Visual Studio .NET
- Microsoft Visual C#®

Summary: Learn about the differences between C# and Java. (68 printed pages)

Whether you are a desktop applications developer or a developer of applications and Web services for Microsoft® Windows® Embedded devices, this technical article will compare and contrast the C# and Java programming languages from an application developer's point of view. This white paper, downloadable from the right-hand corner of this page, describes specifically what is similar, what is different, and the motivation behind the language syntax. It includes side-by-side keyword and code snippet example tables, with a complete usage analysis. It assumes that the reader has some knowledge about C# and/or Java, although it is sufficient to know C++, because both of these languages have C++ similarities, and C++ is often used for comparison.

Contents

Introduction
What's Similar Between C# and Java?
What's Different Between C# and Java?
C# and Java Keyword Comparison
"Interesting" Built-in Class Support
Hits and Misses
Conclusion
Special Thanks

Introduction

Many programming languages exist today: C, C++, Microsoft Visual Basic®, COBOL, C#, Java, and so on. With so many languages, how does a software engineer decide which one to use for a project? Sometimes, a language is chosen because the developers of a company like it or know it, which may be reasonable. Sometimes a language

is used because it is the latest and greatest, and this becomes a marketing tool to generate more public-relations interest in a product, which may not be reasonable in and of itself. In an ideal world, a programming language should be chosen based on its strengths for performing a certain task—the problem to solve should determine the language to use.

This paper would quickly become a book or series of books if it attempted to compare strengths and weaknesses, platform support, and so on for many programming languages. Rather, to limit its scope, it will compare only C# and Java. Some languages, such as C++ and Pascal, will also be used for comparison, but only to help demonstrate potential motivations for the creation of the newer programming languages with their newer features. If some weakness exists and is exposed in the older language, and then shown to be nonexistent or hidden in the newer language, this may help understand the motivation for some change made by the architects of the newer language. Knowing this motivation is often important, because otherwise it is not possible to objectively critique a language.

For example, if a so-called "feature" that existed in a previous language is removed from the newer language, then a developer may feel that the latter is not a worthy candidate to use because it doesn't have the power of the former. This may not be the case; the newer language may be actually doing him a favor by saving him from falling into some known trap.

Naturally, Java came before C#, and C# was not created in a vacuum. It is quite natural that C# learned from both the strengths and weaknesses of Java, just as Java learned from Objective-C, which learned from C. So, C# should be different than Java. If Java were perfect, then there would have been no reason to create C#. If C# is perfect, then there is no reason to create any new programming language. The job would then be done. However, the future is unclear, and both C# and Java are good object-oriented programming languages in the present, so they beg to be compared.

It is important to note that not everything can be covered here. The subject matter is simply too large. The goal is to give enough information so as to help managers and software developers make a better-informed choice about which language to use in certain situations. Maybe some little language quirk in C# may make someone choose Java. Maybe some blemish in Java will influence someone to pick C#. Either way, this document will attempt to dive deep enough into details to dig up some hidden treasures that aid in our goal.

(Items in italics such as the paragraph below and subsequent sections have been highlighted to separate my opinion from the rest of the paper.)

While I am not covering everything about C# and Java in this paper, I will attempt to supply some in-depth analysis on most of the topics that are covered. I don't believe that it is generally worthwhile just to say that some functionality exists in a language and therefore try to imply that the language is powerful. For example, I could simply say, "C# is a good language because it is object oriented." Naturally, that would assume that an object-oriented language is automatically good, which, from my experience, not everyone agrees with. So, I feel in this case that I have to show why writing object-oriented code is good first, which should strengthen the above assertion. This can get a little tedious, but I think that it is important.

Also, I generally do not like to promote anything that I have not used. If I say below that the "language interoperability using Visual Studio .NET is outstanding because it is very easy," then I have run at least some basic tests to really see if it is in fact "easy." More than likely, while not everyone will agree with my opinions, I don't just "wave my hand" by just restating what others have said; rather I try to put what others have said to the test.

What's Similar Between C# and Java?

C# and Java are actually quite similar, from an application developer's perspective. The major similarities of these languages will be discussed here.

All Objects are References

Reference types are very similar to pointers in C++, particularly when setting an identifier to some new class instance. But when accessing the properties or methods of this reference type, use the "**.**" operator, which is similar to accessing data instances in C++ that are created on the stack. All class instances are created on the heap by using the *new* operator, but *delete* is not allowed, as both languages use their own garbage collection schemes, discussed below.

It should be noted that actual pointers may be used in C#, but they can only be manipulated in an unsafe *mode, which is discouraged. This paper will not deal with writing "unsafe" C# code not only because it is discouraged, but also because I have very little experience using it; maybe even more important, because the comparisons with Java will nearly vanish as Java does not support anything like it.*

Garbage Collection

How many times have you used the *new* keyword in C++, then forgot to call *delete* later on? Naturally, memory leaks are a big problem in languages like C++. It's great that you can dynamically create class instances on the heap at run time, but memory management can be a headache.

Both C# and Java have built-in garbage collection. What does this mean? Forgetaboutit! At least forget about calling *delete*. Because if you don't forget, the compiler will remind you! Or worse, Tony might make you a Soprano. Don't be a wise guy; neither language gives you the permission to whack any object that's become expendable. But you may be asked to call *new* fairly often, maybe more than you'd like. This is because all objects are created on the heap in both languages, meaning that the following is frowned on in either language:

```
class BadaBing
{
   public BadaBing()
   {
   }
}

BadaBing badaBoom(); //You can't create temporary data
                     //but you must use parens on a constructor
```

The compiler will send a little message to you about this because you are attempting to create temporary storage. See, there's this thing you gotta do:

```
BadaBing badaBoom = new BadaBing();
```

Now `badaBoom` is made and has at least one reference. Later on, you might be tempted to get rid of him yourself:

```
delete badaBoom; //illegal in C# and Java - the compiler will complain
```

Use `badaBoom` as long as you want, then the garbage collector will dispose of him for you when you decide to give someone else your reference. Waste management is a beautiful thing.

Many developers complain about garbage collection, which is actually quite unfortunate. Maybe they want the control—"I'm gonna kill that puppy off *now*!" Maybe they feel that they're not "real" programmers if they can't delete an object when they created it. Maybe having a more complex and error-prone language guarantees code ownership by the original developer for a longer period of time. Regardless of these reasons, there are some real advantages to garbage collection, some of them being somewhat subtle:

1. No memory leaks. This is the obvious advantage. Both garbage collection schemes promise to dispose of all objects at some point during program execution, but neither can guarantee when, except that no object shall be removed until at least all program references to it are removed.
2. It rewards and encourages developers to write more object-oriented code. This is the subtle advantage. For example, in C++, developers creating class methods that must return data usually either set non-const reference or pointer parameters during method execution; or return a class instance of another type that holds a composite of all necessary data. I consider the latter "better" than the former. Who wants to call a function with 10 parameters? And more parameters passed between client and server code causes more coupling, which is not good. For example, if it is determined later that a function needs to return one more piece of data, then only this function's implementation needs to be changed with an addition to the composite class, which may only require a recompile for the client. Not only that, but when a function returns only one object, this function can be nested with other function calls, while returning data with in/out parameters disallows this nesting. When objects are returned with this "better" method, usually the developer once again has only two choices: return a copy of temporary data initialized and manipulated in the function; or create a new object pointer in the function, side-effect its de-referenced values, then return the pointer, which is safe since the compiler will not destroy pointers or heap data across functions calls. While returning a pointer has some advantages (a copy constructor won't have to be called so it may be faster, particularly with large data; a subclass of a pointer type can actually be returned to the caller for extendibility; and so on), it also has a serious disadvantage in C++: the client will now have to be concerned with memory management by eventually calling *delete* on the returned pointer.
   There are ways around this, one of them being to implement reference counting using generic templates. However, reference counting is not completely "visually" satisfying because of the template syntax; and most if not all counting implementations do not handle cycles correctly while both C# and Java garbage collection schemes do. (Here's a cycle example using simple reference counting: if two objects reference each other, and then only all outside references are released on both, neither will be deleted because they each still have one reference—namely each other—and an object is not deleted until its reference count reaches zero.) Therefore, developers generally take the safe approach, and just return a copy of a compile-time known class type.

   In contrast, because both C# and Java use garbage collection, developers are encouraged to return a *new* reference to data when writing function prototypes (instead of using in/out parameters, for example), which also encourages them to actually return a subclass of the defined return type, or a class instance implementing some interface, where the caller doesn't have to know the exact data type. This allows developers to more easily change service code in the future without breaking its clients by later creating specialized returned-type subclasses, if necessary. In this case the client will only be "broken" if the public interfaces it uses are later modified.

3. Garbage collection makes data sharing easier. Applications are sometimes built which require object sharing. Problems can arise in defining responsibilities for clean-up: if object A and object B share pointer C, should A delete C, or should B? This eliminates the problem: neither A nor B should (or could) delete C using either C# or Java. Both A and B use C as long as they need to, then the system is responsible for disposing of C when it is no longer referenced by either (or any other). Naturally, the developer still must be concerned about critical sections while accessing shared data, and this must be handled in some standard way.

4. Programs should automatically become more "correct." With less to worry about, application developers can concentrate on program logic and correctness rather than memory management, which should create less "buggy" code. This benefit is sometimes understated; it is very important.
5. I am quite sure that there are other advantages that I can't even dream up right now.

Both C# and Java are Type-Safe Languages

Saraswat states on his Web page: "A language is type-safe if the only operations that can be performed on data in the language are those sanctioned by the type of the data." So, we can deduce that C++ is not type-safe according to this definition at least because a developer may cast an instance of some class to another and overwrite the instance's data using the "illegal" cast and its unintended methods and operators.

Java and C# were designed to be type-safe. An illegal cast will be caught at compile time if it can be shown that the cast is illegal; or an exception will be thrown at runtime if the object cannot be cast to the new type. Type safety is therefore important because it not only forces a developer to write more correct code, but also helps a system become more secure from unscrupulous individuals. However, some, including Saraswat, have shown that not even Java is completely type-safe in his abstract.

Both C# and Java Are "Pure" Object-Oriented Languages

Any class in either language implicitly (or explicitly) subclasses an object. This is a very nice idea, because it provides a default base class for any user-defined or built-in class. C++ can only simulate this support through the use of void pointers, which is problematic for many reasons, including type safety. Why is this C# and Java addition good? Well, for one, it allows the creation of very generic containers. For example, both languages have predefined stack classes, which allow application code to *push* any object onto an initialized stack instance; then call *pop* later, which removes and returns the top object reference back to the caller—sounds like the classic definition of a stack. Naturally, this usually requires the developer to cast the popped reference back to some more specific object-derived class so that some meaningful operation(s) can be performed, but in reality the type of all objects that exists on any stack instance should really be known at compile-time by the developer anyway. This is at least because it is often difficult to do anything useful with an object if the class's public interface is unknown when later referencing some popped object. (*Reflection*, a very powerful feature in both languages, can be used on a generic object. But a developer would be required to strongly defend its use in this scenario. Since reflection is such a powerful feature in both languages, it will be discussed later.)

In C++, most developers would use the *stack* container adapter in the Standard Template Library (STL). For those unfamiliar with the STL, Schildt (p. 5) states that it was designed by Alexander Stepanov in the early 1990s, accepted by the ANSI C++ committee in 1994 and available in most if not all commercial C++ compilers and IDEs today, including the eMbedded Visual Tools. Sounds like an encyclopedia's version of reality. Essentially, it is a set of containers, container adapters, algorithms, and more, simplifying C++ application development by allowing any C++ class (and most primitives) to be inserted and manipulated through a common template definition. Sounds like a nifty idea.

However, there are issues with templates in C++. Say that a new `stack` of `ints` is created by:

```
#include <stack>
using namespace std;
stack<int> intStack;
```

The template definition is found, then an actual `int` stack class definition and implementation code are created implicitly behind-the-scenes, using that template. This naturally adds more code to the executable file. OK; so maybe it's not a big deal, really. Memory and hard drive space is cheap nowadays. But it could be an issue if a developer uses many different template types. From a purist's perspective, it could be viewed as wasteful to have a new `stack` class definition created and compiled for every type that a `stack` holds. A stack is a stack: it should only have a constructor, a destructor, a "push", a "pop", and maybe a Boolean "empty" and/or "size" method. It doesn't need anything else. And it shouldn't care what type it holds, in reality, as long as that type is an object.

The STL's stack, however, doesn't work this way. It requires compile-time knowledge of what type it will hold (the template definition doesn't care, but the compiler does). And it doesn't have the "classic" set of stack functions. For example, the *pop* is a *void* function; the developer must first call *top*, which returns an address to the top of the stack, then call *pop*, which means that a single operation has now become two! The inherent problem in C++ that most likely motivated this awkward decision: if the `pop` function returned an element and removed it from the `stack`, it would have to return a copy (the element's address would no longer be valid). Then if the caller decides he doesn't want the element after inspection, he would then have to *push* a copy back on the `stack`. This would be a slower set of operations, particularly if the type on the `stack` was quite large. So a `top` or inspect method was added which would not side-effect the number of `stack` elements, allowing the developer to peek at a class instance before he removes it. But when a developer does access this top element, then calls some function on it, he may be side-effecting an element in the container, which may at least be bad style in some scenarios. The original STL architecture could have required that developers only use pointers with containers (in fact, pointers are allowed, but you still need to be concerned with memory management), which might have influenced a prototype change causing the `pop` method to remove and return a pointer to the top element, but then the lack-of-garbage-collection issue returns. It appears as if the limitations and problems inherent to C++ required changing the STL's public `stack` definition from the "classic" view, which is not good. Why is this not good? One example that I am personally familiar with: the first time that a developer reads the specification for an STL `stack`, he is sad, for one. The first time that he uses the implementation and calls `top` but forgets to call pop, and gets mad, for two. The only arguable advantage in C++ using stack templates compared to C#'s or Java's stack: no casting is required on the template's `top` method because the compile-time created `stack` class has knowledge about what address type it must hold. But this is truly minor if the assertion above convinces a developer that he should know what type is allowed on his particular `stack` instance anyway.

In contrast, the public interfaces for the stack classes in both C# and Java follow the classic paradigm: `push` an object on the stack, which places this object on the top; then `pop` the stack, which removes and returns the top element. Why can C# and Java do this? Because they are both OOP languages, and perhaps more important, because they both support garbage collection using references. *The stack code can be more aggressive because it knows that the client won't have to handle cleanup.* And a stack can hold any object, which any class must be implicitly.

When I code in C++, I use the STL as much as possible, which may seem strange since I seem to complain about it so much above. In my opinion, the STL is a "necessary evil" when using C++. I once tried to build some C++ framework that was "better" than the STL by playing around with my own stack class, using pointers, inheritance, and memory management, just for fun. Suffice it to say that I was unable to do better than the STL, mostly due to problems dealing with destruction. The STL actually works very nicely within the limitations of C++.

Standard C++ would have been a much better language if only one addition had been made: make every class implicitly an object. That's it! Garbage collection is very nice, but not completely necessary. If every class were implicitly an object in C++, then a virtual *destructor method could exist on the object base, and then generic C++ containers could have been built for any class where the container handles memory management internally. For example, a stack class could be built, which holds only object pointers. When the stack's destructor is eventually called,* delete *could be called on any object left on the stack, implicitly calling the correct destructor. Methods on this stack could be created that would allow either a* pop *to remove the top pointer element and return it, which would require the developer to be in charge of its later destruction; or return a copy of the element then have the stack* delete *the pointer internally, for example. Templates would then be unnecessary.*

It may be worth your while to look into managed C++ *code. I have played with it a little, and discuss at least what I know briefly below. Managed C++ code uses "my" recommendation (make every class instance an object) but also includes memory management. It's actually pretty good.*

There are many other reasons why it is preferable to use a "pure" object-oriented language, including application extensibility and real-world modeling. But what defines a "pure" object-oriented language anyway? Ask five separate people and you'll most likely get five wrong answers. This is because the requirements of a "pure" object-oriented are fairly subjective. Here's probably the sixth wrong answer:

- Allows the creation of user-defined types, usually called a *class*
- Allows the extension of classes through inheritance and/or the use of an *interface*
- All user-created types implicitly subclass some base class, usually called *object*
- Allows methods in derived classes to override base class methods
- Allows casting a class instance to a more specific or more general class
- Allows varying levels of class data security, usually defined as *public*, *protected* and *private*
- Should allow operator overloading
- Should not allow or highly restricts global function calls—functions should rather be methods on some class or interface instance
- Should be type-safe—each type has some data and a set of operations that can be performed on that type
- Has good exception-handling capabilities
- Arrays should be first-class objects: a developer should be able to query one on its size, what type(s) it will hold, and more

Since C++ was originally designed to be compatible with C, it fails at being a "pure" object-oriented programming immediately because it allows the use of global function calls, at least if applying the requirements described above. And arrays are not first-class objects in C++ either, which has been the cause of great agony in the developer world. Naturally, if a developer uses a subset of the C++ specification by creating array wrappers, using inheritance, avoiding global function calls, and so on, then his specific C++ code could arguably be called object-oriented. But because C++ allows you to do things that are not allowed in our "pure" object-oriented language definition, it can at best be called a hybrid.

In contrast, both C# and Java seem to meet the above criteria, so it can be argued that they are both "pure" object-oriented programming languages. Is the above minimum requirement criterion set legitimate? Who knows. It seems as if only real-world programming experience will tell you if a language is truly object-oriented, not necessarily some slippery set of rigid requirements. But some set of rules must be established to have any argument at all.

Single Inheritance

C# and Java allow only single inheritance. What's up with that? Actually, it's all good—any class is allowed to implement as many interfaces as it wants in both languages. What's an *interface*? It's like a class—it has a set of methods that can be called on any class instance that implements it—but it supplies no data. It is only a definition. And any class that implements an interface must supply all implementation code for all methods or properties defined by the interface. So, an interface is very similar to a class in C++ that has all pure *virtual* functions (minus maybe a *protected* or *private* constructor and *public* destructor which provide no interesting functionality) and supplies no additional data.

Why not support multiple inheritance like C++ does? Lippman (p. 472) views the inheritance hierarchy graphically, describing it as a directed acyclic graph (DAG) where each class definition is represented by one node, and one edge exists for each base-to-direct-child relationship. So, the following example demonstrates the hierarchy for a Panda at the zoo:

What's wrong with this picture? Well, nothing, as long as only single inheritance is supported. In the single inheritance case, there exists only one path from any base class to any derived class, no matter the distance. In the multiple inheritance case there will be multiple paths if, for any node, there exists a set of at least two base classes

which share a base of their own. Lippman's example (p. 473) is shown below.

In this case, it is common for a developer to explicitly use *virtual* base classes, so that only one memory block is created for each class instantiation no matter how many times it is visited in the inheritance graph. But this requires him to have intimate knowledge of the inheritance hierarchy, which is not guaranteed, unless he inspects the graph and writes his code correctly for each class that he builds. While it should be possible for a developer to resolve any ambiguity that may arise by using this scheme, it can cause confusion and incorrect code. [1]

In the single-inheritance-multiple-interface paradigm, this problem cannot happen. While it is possible for a class to subclass two or more base classes where each base implements a shared interface (if class `One` subclasses class `Two`, and `Two` implements interface `I`, then `One` implicitly implements `I` also), this is not an issue, since interfaces cannot supply any additional data, and they cannot provide any implementation. In this case, no extra memory could be allocated, and no ambiguities arise over whose version of a *virtual* method should be called because of single inheritance.

So, did C# and Java get it right, or did C++? It depends upon whom you ask. Multiple inheritance is nice because a developer may only have to write code once, in some base class, that can be used and reused by subclasses. Single inheritance may require a developer to duplicate code if he wants a class to have behavior of multiple seemingly unrelated classes. One workaround with single inheritance is to use interfaces, then create a separate implementation class that actually provides functionality for the desired behavior, and call the implementer's functions when an *interface* method is called. Yes, this requires some extra typing, but it does work.

Arguing that single inheritance with interfaces is better than multiple inheritance strictly because the former alleviates any logic errors is unsound, because a C++ developer can simulate interfaces by using pure *virtual* classes which provide no data. C++ is just a little more flexible. Arguing the contra position is also unsound, because a C# or Java developer can simulate multiple inheritance by the methods discussed above, granted with a little more work. Therefore, it could be argued that both schemes are equivalent.

Theory is great, but while coding user interfaces in Java, I ran into situations where I really could have used multiple inheritance. Since all items displayed to screen must subclass java.awt.Component*, custom components that I built were required to cast to this base since Java uses single inheritance. This required me to use interfaces extensively for any additional general behavior that these components needed to implement, which worked, but was tricky and required much typing.*

From this experience I learned that it is often good to be conservative in the single-inheritance world. When building user interfaces, you don't have much choice—you must subclass some predefined base. But during the development of most code, think long and hard before creating some abstract base class for your classes to extend, because once a class extends another, it now can only implement interfaces. While it is possible to eventually toss an abstract base class, define its methods in an interface, then redo classes which previously subclassed this base and now implement an interface, it can be a lot of work.

The lesson: if you feel that an abstract base class is necessary because this class requires quite a few methods, and most of these methods can be implemented in the base with potentially few subclass overrides, then feel free to do it. But if you find that the abstract base has very few methods, and maybe many of these methods are abstract too since it is unclear what the default behavior should be, then it may be best to just define and implement an interface. This latter method will then allow any class later on that implements the interface to subclass any other that it chooses.

Built-in Thread and Synchronization Support

Languages such as ANSI C++ give no support for built-in threading and synchronization support. Third-party packages that supply this functionality must be purchased, based on operating system, and the APIs vary from package to package. While development tools such as Visual Studio 6 supply APIs for C++ threading, these APIs aren't portable; they generally can only be used on some Microsoft operating system.

Both C# and Java, however, both have built-in support for this functionality in their language specifications. This is important because it allows a developer to create multi-threaded applications that are immediately portable. It's usually "easy" to build a portable, single-threaded C++ application; try building a portable C++ multi-threaded app, particularly some graphical user interface (GUI) app. And most applications nowadays are multi-threaded, or if they aren't, they should be. But in C# and Java, a developer can use the built-in language thread and synchronization functions and feel very secure that programs will run in a similar fashion on different platforms, or at least similar enough to guarantee correctness.

Java and C# differences and similarities will be explained in more detail later.

Formal Exception Handling

Formal exception handling in programming languages generally supplies a developer with program flow control during exceptional runtime conditions. This is supplied with the ability to *throw* an exception from a function if anything "bad" occurs during execution. It also supplies any application calling this function the ability to *try* and *catch* a potential error, and optionally *finally* do something after a method call no-matter-what. When a function *throws* an exception, no further code in the throwing function is executed. When a client handles the exception, no further code in the *try* block of a try-catch [finally] (TCF) statement is executed. The following C# example demonstrates some basics.

```
using System;

namespace LameJoke
{
   //Stones is an instance of an Exception
   public class Stones : Exception
   {
      public Stones(string s) : base(s)
      {

      }
   }

   //All instances of People are poorly behaved - Creation is a failure
   public class People
   {

      /**
      * Throws a new Stones Exception.  Shouldn't really do this in a
      * constructor
      */
      public People()
      {
         throw (new Stones("Exception in constructor is bad"));
```

```
      }
   }

   //All GlassHouses fail during construction
   public class GlassHouses
   {
      //private data member.
      private People m_people = null;

      //Throws an Exception because all People throw Stones
      public GlassHouse()
      {
         m_people = new People();
      }
   }

   //Main function
   static void Main()
   {
      GlassHouses glassHouses = null;

      //try-catch-finally block
      //try - always exectuted
      //catch - executed only if a Stones exception thrown during try
      //finally - always executed
      try
      {
         glassHouses = new GlassHouses();
      }
      catch (Stones s)
      {
         //This block is executed only if all People are poorly
         //behaved
         . . .
      }
      finally
      {
         //. . .it's nearly over. . .
      }

      //glasshouses is still null since it failed to be constructed
   }
```

```
}
```

Both C# and Java have support for formal exception handling, like C++. Why do people feel the need for exception handling, though? After all, languages exist that do not have this support, and developers are able to write code with these languages that works correctly. But just because something works doesn't mean that it's necessarily good. Creating functions using formal exception handling can greatly reduce code complexity on both the server and client side. Without exceptions, functions must define and return some invalid value in place of a valid one just in case preconditions are not met. This can be problematic since defining an invalid value may remove at least one otherwise valid item in the function range. And it can be messy because the client must then check the return value against some predefined invalid one. (Other solutions have been tried, among them: 1) add an extra non-const Boolean reference to every function call, and have the method set it to true if success else false. 2) Set a global parameter, for at least the calling thread's context, which defines the last error that a client can test after function calls. These are far from satisfying, and they may require an application developer to have too much knowledge about how things work "under the covers.")

Look at the *System.Collections.Stack* class supplied in Microsoft's .NET Framework, or Java's java.util.Stack class which is very similar. Both seem to be designed reasonably: a `void Push(Object)` method, and an `Object Pop()` method. The latter function throws an Exception if it is called on an empty `Stack` instance, which seems correct. The only other reasonable option is to return *null*, but that is messy, because it requires the developer to test the validity of the returned data to avoid a null pointer exception. And popping an empty `Stack` should be an invalid operation anyway, because it means that `Pop` has been called at least once more than `Push`, implying that the developer has done a poor job of bookkeeping. With .NET's `Stack` prototype and implementation, coupled with the exception handling rules in C# (C# does not require an exception to be caught if the developer knows that all preconditions have been met; or, if sadly, he could care less—or is that supposed to be "couldn't care less?". . .), there are several options:

```
Stack s = new Stack();
s.Push(1);
int p = (int)s.Pop();
```

In the previous case, the developer knows that an exception cannot be thrown from the `Pop` method, because the `Push` call has been made previously on a valid object and no Pop has yet been performed. So he chooses "correctly" to avoid a TCF statement. In comparison:

```
try
{
   Stack s = new Stack();
   s.Push(1);
   int p = s.Pop();
   p = s.Pop();
}
catch (InvalidOperationException i)
```

```
{
   //note:  this block will be executed immediately after the second Pop
   . . .
}
```

In this next case, for some reason the developer is not convinced that `Push` is called at least as many times as `Pop` on `Stack s`, so he chooses to catch the possible Exception type thrown. He has chosen wisely. However, in reality, a client really should make sure that he doesn't call `Pop` more often than he calls `Push`, and he really shouldn't verify this by lazily catching some Exception—it's at the very least considered "bad style" in this particular case. But the `Stack` code cannot guarantee that the application using it will behave correctly, so the `Stack` can just throw some Exception if the application behaves poorly. Not only that, if exception handling were not used or available, what *should* the `Stack` code return if the client tries to `Pop` an empty `Stack`? *null*? The answer is not completely clear. In some languages like C++, where a data copy is returned from functions in most implementations, *NULL* is not even an option. By throwing an exception, the `Stack` code and its clients do not have to negotiate some *return* value that is outside the range of accepted values. The stack's `Pop` method can just exit by throwing an exception as soon as some precondition is not met, and code in the first *catch* block on the client side which handles a castable type to the exception thrown will be entered. Yes, even exception handling uses an object-oriented approach! Code inside a TCF statement works nicely together, as there is an implied set of preconditions stating that no subsequent line in the block will be executed in case some previous line causes some exception to be thrown. This makes logic simpler as the client is not required to test for error conditions before executing any line of code in the block. And the class code can just bail out of a method immediately by throwing an exception if anything "bad" happens, and this exception implies that there will not be a valid *return* item.

It should be noted that not all functions should throw exceptions. The *Math.Cos* function, for example, is defined over the set of all reals. For what value will the function throw an Exception? Maybe infinity. Negative infinity? It would be great if all functions were as well-behaved as `Cos` because exception handling would consume about zero percent of all code. In comparison, the *Acos* function is only defined for all reals between -1 to 1, inclusive. But it's not completely clear how an `Acos` function should handle an out-of-range value. Throw an Exception? Return an invalid value? Take a nap? The *System.Math* class's version in the .NET Framework returns a *NaN* value (not a number) but it could just as easily throw an Exception. But can you imagine having to use TCF statements every time a math function is called? Yuk. Any application code littered with TCF statements can get ugly in a hurry as well. What's the moral of the story? Exception handling is great, but it should be used with care. Oh, and `People` in `GlassHouses` should not throw `Stones`—at least not during construction. Maybe it's a recommendation that I should heed more often. . . .

Built-in Unicode Support

Both C# and Java use only Unicode, which greatly simplifies internationalization issues by encoding all characters with 16 bits instead of eight. With the proper use of other internationalization manager classes along with properties files, an application can be built which initially supports English but can be localized to other languages with no code change by simply adding locale-specific properties files.

This feature is very important now, and will become vitally important in the future. Assuming that your application will only run in English is almost always unacceptable nowadays. This subject really deserves more explanation, but unfortunately time constraints force us to put it off for another day. Suffice it to say that both C# and Java have excellent support for internationalization. I should know; I have used them both.

I, probably like you, visit Web sites almost every day that have no English support. If I really need some information, and I can't understand the text, then I'm pretty disappointed if the site appears to have the information that I need. Trying to understand these Web sites has given me more empathy for those who don't speak English.

I have built a few ASP.NET Web applications for "fun," one of them a C# version for a demo showing how one Web site could be built that could display on almost any OS with almost any browser, with multiple language support. It was very nice when it displayed correctly on a Windows CE device, and it was awesome when I realized that the JavaScript, written behind-the-scenes for me by Visual Studio .NET, was even correct so that my application would work correctly for users with Netscape! Naturally, the built-in i18n support made things pretty simple.

You may want to look into using these Web applications with ASP.NET Web services yourself. They work very well and really make your job easier. Oh, and your friends abroad will thank you for it to by spending money on your site.

What's Different Between C# and Java?

While C# and Java are similar in many ways, they also have some differences. If they didn't, there would have been no reason to develop C# at all, since Java has been around longer.

Formal Exception Handling

Wait a minute. It said above that both languages have formal exception handling in the "what's similar" section. And now it's in the "what's different section." So which one is it? Similar or different? Different or similar? Well, they are very similar, but there are some important differences.

For starters, Java defines a java.lang.RuntimeException type, which is a subclass of the base class of all exceptions, java.lang.Exception. Usually, RuntimeExceptions are types that are thrown when a client will be able to test preconditions easily, or implicitly knows that these preconditions will always be met before making a method call. So, if he knows that a RuntimeException should not be thrown from the method, then he is not required to catch it. Simply stated: Java expects Exception acceptance except RuntimeExceptions by expecting Exception inspection using explicit expert exception handling.

This just makes no sense at all. Why does Java clearly differentiate between Exceptions and RuntimeExceptions? Maybe what you really want to know: "How will I know when I must use a TCF statement? When I should? When I shouldn't?" The compiler may give you a hint in some situations. Any method that may throw an `Exception` must include all possible `Exception` types by name in the method's *throws* list (a `RuntimeException` subclass is optional in this list, although it would be wise to add it there). The client calling the method must use a TCF statement if the method called may throw at least one non-`RuntimeException` Exception (let's call it an NRE to make things clear, which should be the goal of any good document). The following mysterious example demonstrates some syntax and rules:

```
public class Enigma
{
   public Enigma()
   {
      //Do I really exist?
   }

   public void Riddle()
   throws ClassNotFoundException,
          NoSuchMethodException,
          IllegalArgumentException
```

```
    {
        //do something here
    }
}
```

Assuming that a client can call an existing *public* method with an empty formal parameter list on a properly initialized object instance, and this method might unexpectedly throw a NoSuchMethodException, an IllegalArgumentException or a ClassNotFoundException (hey, stuff happens):

```
Enigma enigma = new Enigma();

try
{
   enigma.Riddle();
}
catch (ClassNotFoundException c)
{
   //do something here
}
catch (NoSuchMethodException n)
{
   //do something here
}
```

Since all throwables in Java and C# must subclass `Exception`, it should be noted that the client could simply use one *catch* block that catches the base type: `Exception`. This is completely acceptable if he will do the exact same thing no matter what "bad" thing happens during the method call, or if some other class developer who moonlights as a comedian decides to implement a method that *throws* 27 Exception types. It should also be noted that catching the `IllegalArgumentException` is optional, as this type extends a `RuntimeException`. He probably made the right call by not catching the `IllegalArgumentException`, because his vast array of parameters should be valid in this case.

In contrast, C# disallows the use of a *throws* list. From a client's perspective, C# treats all Exceptions like Java's RuntimeExceptions, so the client is never required to use a TCF statement. But if he does, the syntax and rules of C# and Java are nearly if not exactly identical. And like Java, every item that is thrown must be castable to `Exception`.

So, which system is "better?" The exception handling rules of Java, or the exception handling rules of C#? They are equivalent, because any MS eMVP or MVP using C# with the Visual Studio .NET IDE and CLR running on XP with SP1 can simulate J# NREs using TCFs. It's that simple (ITS). So deciding which scheme is "better" is completely subjective. One completely subjective view is given below.

It is my opinion that the Java rules of exception handling are superior. Why? Requiring a throws *list in a method definition clearly signals to the client developer what exceptions he may catch, and helps a compiler help this developer by explicitly dictating which Exceptions must be caught. With C#, the only way for the developer to know which Exceptions may be thrown is by manually inspecting documentation, pop-up help, code or code comments. And it may be difficult for him to decide in which scenarios it is appropriate to use a TCF statement. Exceptions that may be thrown from a method really should be included in the "contract" between client and server, or the formal method definition or prototype.*

Why did C# choose not to use a throws *list, and never require a developer to catch any* Exception*? It may be due to some interoperability issues between languages such as C# and C++. The latter, which actually defines a* throw *list as optional on a class method in a specification file, likewise does not require a C++ client to catch any "Exception" (any class or primitive can be thrown from a C++ function—it does not have to subclass some Exception class—which was a poor design decision). And J# does not require a client to catch any* Exception*, like C#. Java is a language which generally is used by itself, while C#, C++, and any other language supported now or in the future using Visual Studio .NET, are supposed to work together when using managed code. Or it could be that C# architects simply believe that using TCFs should always be optional to a client. I have heard one theory that the original motivation was so that server code could be modified to throw different exception types without modifying client code, but that seems slightly dangerous to me in case the client does not ever catch the base exception.*

At any rate, thumbs up from me to Java on this one.

Java Will Run on "Any" Operating System

One of the original motivations for creating Java was to create a language where compiled code could run on any operating system. While it is possible in some situations to, say, write portable C++ code, this C++ source code still needs to be compiled to run on some new targeted operating system and CPU. So Java faced a large challenge in making this happen.

Compiled Java does not run on "any" operating system, but it does run on many of them. Windows, UNIX, Linux, whatever. There are some issues with Java running on memory-constrained devices as the set of supported libraries must be reduced, but it does a good job of running on many OSes. Java source code is compiled into intermediate byte-codes, which are then interpreted at run time by a platform-specific Java Virtual Machine (JVM). This is nice, because it allows developers to use any compiler they want on any platform to compile code, with the assumption that this compiled byte-code will run on any supported operating system. Naturally, a JVM must be available for a platform before this code can be run, so Java is not truly supported on every operating system.

C# is also compiled to an intermediate language, called MSIL. As the online documentation describes, this MSIL is converted to operating system and CPU-specific code, generally a just-in-time compiler, at run-time. It seems that while MSIL is currently only supported on a few operating systems, there should be no reason that it cannot be supported on non-Windows operating systems in the future.

Currently, Java is the "winner" in this category as it is supported on more operating systems than C#.

One good thing about both the approaches that C# and Java have taken: they both allow code and developers to postpone decision making, which is almost always a good thing. You don't have to exactly know what operating system that your code will run on when you start to build it, so you tend to write more general code. Down the road, when you find out that your code needs to run on some other operating system, you're fine as long as your compiled byte-code or MSIL is supported on that platform. If you originally assumed that your code was to run on only one OS and you took advantage of some platform-specific functionality, then later determine that your code must run on some other OS, you're in trouble. With both C# and Java, many of these problems will be avoided.

Naturally, if you are going to build applications, you need to know what operating systems your code will run on so that you can meet your customer's needs. But in my opinion, you don't necessarily have to know every dirty little detail about how the JVM works, for example, just that it does. Sometimes, ignorance can be bliss.

C# and Java Language Interoperability

While others like Albahari have broken interoperability into different categories such as language, platform, and standards, which is actually quite interesting to read, only language interoperability will be discussed in this paper due to time constraints. C# is a winner in this category, with one current caveat: any language targeted to the CLR in Visual Studio .NET can use, subclass, and call functions only on managed CLR classes built in other languages. While this is possible, it is no doubt more awkward in Java, as in other programming languages not yet supported in .NET.

Grimes (p. 209) describes how "incredibly straightforward" it is to create Java code for a Windows-based operating system that can access COM objects. Inspecting his code, I do agree that the client code can be fairly simple to implement. But you still have to build the ATL classes, which can be some work.

In contrast, using Visual Studio .NET, libraries can be built in J#, Visual Basic .NET, and managed C++ (with other languages to come) and subclassed or directly used in C# with extreme ease. Just looking at the libraries available to you in the online documentation, the developer has no idea if the classes were built in C++, C# or another. And he doesn't really care anyway, because these all work so nicely together.

I decided to test how easy language interoperability is using Visual Studio .NET. So, I created a C++ managed code library by using the following procedure:

1. *Open Visual Studio .NET.*
2. *On the **File** menu, click **New Project**.*
3. *In the left pane, click **Visual C++ Projects**.*
4. *In the **Templates** pane on the right, click **Managed C++ Class Library**.*
5. *Set the **Name** and **Location** of the project.*
6. *Click **OK**.*

The project created one class automatically, called "Class1" (nice name!) as follows:

```
public __gc class Class1
```

The online documentation refers to this `__gc` *as a* managed pointer. *These are very nice in C++, because the garbage collector will automatically destroy these types of C++ objects for you. You can call delete if you want, but from some testing that I performed, you don't have to explicitly. Eventually, the destructor will be called implicitly. Wow —C++ with garbage collection! Nifty. (Something else that is "nifty:" you can also create properties with managed C++ code by using the* `__property` *keyword, where these properties behave similarly to C#'s version below.)*

I defined and implemented a few simple member functions, compiled my library, then created a C# Windows application by using the following procedure:

1. *Open Visual Studio .NET (another instance, if you want).*
2. *On the **File** menu, click **New Project**.*
3. *In the left pane, click **Visual C# Projects**.*
4. *In the **Templates** pane on the right, click **Windows Application**.*

5. *Set the **Name** and **Location** for the project.*
6. *Click **OK**.*

Then I imported the C++ compiled dll (all libraries are now dlls, which is actually good) into my C# project by using the following procedure:

1. *Click **Project**/**Add Reference**.*
2. *Click the **Projects** tab (maybe not necessary? Seemed logical).*
3. *Click **Browse**.*
4. *Search for the dll built by the managed C++ project above and select it.*
5. *Click **OK**.*

I then created an instance of the C++ class in the C# project, compiled my project, and stepped through the code. It is very strange walking through C++ code in a C# project. Very strange but very nice. Everything worked. The C# code seemed to have no idea that the object was created in C++—the way it's supposed to be.

I then created a C# class called Class2 (nice name by me) which subclassed the C++ Class1(!), and implemented a non-virtual method in the latter and a new method with the same signature in the former. Creating a new instance of Class2 and assigning it to a Class1 reference and calling this method on the reference, sure enough, the C++ version was called correctly. Modifying this method to be virtual in the base, and override in the subclass, recompiling, and running the code, sure enough, the C# version was called.

Finally, I installed the J# plug-in for Visual Studio .NET, and created a J# library project similar to the method above for the managed C++ project. I imported the C++ dll into this project, and subclassed Class1 with a Java class, and implemented the virtual function implicitly. I then imported this J# dll into the C# project, ran similar tests with the new Java object, and the correct Java (implicit) virtual method was called in the C# code.

I don't know much about Visual Basic, so I had to leave this for another day. I have written and modified some VBScript, but that is the limit of my knowledge. I apologize to you Visual Basic developers. . . .

It would be fun to play with this for a couple of days, and test to see if everything works correctly. I cannot say that everything does, but it sure is cool, and it sure is easy. I can imagine creating applications with multiple libraries, where each library uses the language that is "most fit" for the specific problem, then integrating all of them to work together as one. The world would then be a truly happier place.

C# Is a More Complex Language than Java

It seems as if Java was built to keep a developer from shooting himself in the foot. It seems as if C# was built to give the developer a gun but leave the safety turned on. And it seems as if when C++ was built, they just handed the programmer a fully loaded bazooka with an open-ended license to use it. C# can be as harmless as Java using safe code, but can be as dangerous as C++ by clicking off that safety in *unsafe* mode—you get to decide. With Java, it seems as if the most damage that you can do is maybe spray yourself in the eye with the squirt gun that it hands you. But that's the way that the Java architects wanted it, most likely. And the C# designers probably wanted to build a new language that could persuade C++ developers, who often want ultimate firepower and control, to buy into it.

Below, I will provide some proof to this argument that "C# is a more complex language than Java."

C# and Java Keyword Comparison

Comparing the keywords in C# and Java gives insight into major differences in the languages, from an application developer's perspective. Language-neutral terminology will be used, if possible, for fairness.

Equivalents

The following table contains C# and Java keywords with different names that are so similar in functionality and meaning that they may be subjectively called "equivalent." Keywords that have the same name and similar or exact same meaning will not be discussed, due to the large size of that list. The Notes column quickly describes use and meaning, while the example columns give C# and Java code samples in an attempt to provide clarity.

It should be noted that some keywords are context sensitive. For example, the *new* keyword in C# has different meanings that depend on where it is applied. It is not used only as a prefix operator creating a new object on the heap, but is also used as a method modifier in some situations in C#. Also, some of the words listed are not truly keywords as they have not been reserved, or may actually be operators, for comparison. One non-reserved "keyword" in C# is *get*, as an example of the former. *extends* is a keyword in Java, where C# uses a ':' character instead, like C++, as an example of the latter.

C# Keyword	Java Keyword	Notes	C# Example	Java Example

Base	super	Prefix operator that references the closest base class when used inside of a class's method or property accessor. Used to call a super's constructor or other method.	`public MyClass(string s) : base(s)`
			`{`
			`}`
			`public MyClass() : base()`
			`{`
			`}`
				`Public MyClass(String s)`
				`{`
				`super(s);`
				`}`
				`public MyClass()`
				`{`
				`super();`
				`}`

Bool	boolean	Primitive type which can hold either true or false value but not both.	`bool b = true;`	`boolean b = true;`
Is	instanceof	Boolean binary operator that accepts an l-value of an expression and an r-value of the fully qualified name of a type. Returns true iff l-value is castable to r-value.	`MyClass myClass = new MyClass();` `if (myClass is MyClass)`	`MyClass myClass = new MyClass();` `if (myClass instanceof MyClass)`

			`{`
			
			`//executed`
			
			`}`
				`{`
				
				`//executed`
				
				`}`

lock	synchronized	Defines a mutex-type statement that locks an expression (usually an object) at the beginning of the statement block, and releases it at the end. (In Java, it is also used as an instance or static method modifier, which signals to the compiler that the instance or shared class mutex should be locked at function entrance and released at function exit, respectively.)	`MyClass myClass = new MyClass();`
			`lock (myClass)`
			
			`{`
			
			`//myClass is`
			
			`//locked`
			
			`}`
			
			`//myClass is`
			
			`//unlocked`
				`MyClass myClass = new MyClass();`
				`synchronized (myClass)`
				
				`{`
				
				`//myClass is`
				
				`//locked`
				
				`}`
				
				`//myClass is`
				
				`//unlocked`

namespace	package	Create scope to avoid name collisions, group like classes, and so on.	**`namespace`** `MySpace`
			`{`
			
			`}`
				`//package must be first keyword in class file`
				`package` `MySpace;`
				
				`public class MyClass`
				
				`{`
				
				`}`

readonly	const	Identifier modifier allowing only read access on an identifier variable after creation and initialization. An attempt to modify a variable afterwards will generate a compile-time error.	`//legal initialization`
			`readonly` `int constInt = 5;`
			
			`//illegal attempt to`
				`//legal initialization`
				`const` `int constInt = 5;`
				
				`//illegal attempt to`
				
				`//side-effect variable`
				
				`constInt = 6;`

			//side-effect variable
			
			constInt = 6;	

sealed	final	Used as a class modifier, meaning that the class cannot be subclassed. In Java, a method can also be declared final, which means that a subclass cannot override the behavior.	//legal definition
			public **sealed** class A
			
			{
			
			}
			
			//illegal attempt to
			
			//subclass - A is
			
			//sealed
			
			public class B: A
			
			{
			
			}
				//legal definition
				public **final** class A
				
				{
				
				}
				
				//illegal attempt to
				
				//subclass - A is
				
				//sealed
				
				public class B extends A
				
				{
				
				}

using	import	Both used for including other libraries into a project.	**using** System;	**import** System;

internal	private	Used as a class modifier to limit the class's use inside the current library. If another library imports this library and then attempts to create an instance or use this class, a compile-time error will occur.	namespace Hidden
			{
			
			internal class A
			
			{
			
			}
			
			}
			
			//another library
				package Hidden;
				private class A
				
				{
				
				}
				
				//another library
				
				import Hidden;
				
				//attempt to illegally

			`using Hidden;`
			
			`//attempt to illegally`
			
			`//use a Hidden class`
			
			`A a = new A();`
				`//use a Hidden class`
				
				`A a = new A();`

:	extends	Operator or modifier in a class definition that implies that this class is a subclass of a comma-delimited list of classes (and interfaces in C#) to the right. The meaning in C# is very similar to C++.	`//A is a subclass of`
			`//B`
			
			`public class A : B`
			
			`{`
			
			`}`
				`//A is a subclass of`
				`//B`
				
				`public class A` **`extends`** `B`
				
				`{`
				
				`}`

:	implements	Operator or modifier in a class definition that implies that this class implements a comma-delimited list of interfaces (and classes in C#) to the right. The meaning in C# is very similar to C++.	`//A implements I`
			`public class A : I`
			
			`{`
			
			`}`
				`//A implements I`
				`public class A`
				`implements` `I`
				
				`{`
				
				`}`

Supported in C# but Not in Java

The following table enumerates keywords in C# that seem to have no equivalent atomic support in Java. If possible or interesting, code will be written in Java that simulates the associated C# support to demonstrate the keyword's functionality in C# for Java developers. It should be noted that this list is very subjective, because it is highly unlikely that two people working independently would arrive at the same comparison list.

C# Keyword	Notes	C# Example	Java Equivalent
as	Binary "safe" cast operator that accepts expression as an l-value and the fully qualified class type as the r-value.	`Object o = new string();` `string s = o` **`as`** `string;`	`Object o = new String();` `string s = null;`

	Returns corresponding reference of r-value type if castable else null.	`if (null != s)`
		`{`
		`//executed`
		`Console.writeln(s);`
		`}`
			`if (o instanceof String)`
			`{`
			`s = (String) o;`
			`}`
			`if (null != s)`
			`{`
			`//executed`
			`System.Out.Writeln(s);`
			`}`

checked	Creates a statement with one block, or unary expression operator. Requires the developer to catch any arithmetic exceptions that occur during block or expression evaluation.	`using System;`
		`short x = 32767;`
		`short y = 32767;`
		`checked`
		`{`
		`try`
		`{`
		`short z = y + z;`
		`}`
		`catch (OverflowException e)`
		`{`
		`//executed`
		`}`	

		`}`	
decimal	Defines a 128 bit number.	`decimal d = 1.5m;`	
delegate	Very similar to a C++ function pointer "on steroids." Because of its complex nature, it will be discussed in more detail below.	`delegate void MyFunction();`	

enum	Very similar to *enum* in C++. Allows a developer to create a zero-relative type with a zero-relative named list. It is too bad that Java chose to not allow enums. They are somewhat important.	`enum colors {red, green, blue};`	`public class Colors`
			`{`
			`public static const Red = 0;`
			`public static const Green = 1;`
			`public static const Blue = 2;`
			`private int m_color;`
			`public Colors(int color)`
			`{`
			`m_color = color;`
			`}`
			`public void SetColor(int color)`
			`{`
			`m_color = color;`
			`}`
			`public int GetColor()`
			`{`
			`return (m_color);`

			`}` `}`
event	Allows a developer to create event handlers in C#. Discussed more below.	`public` **`event`** `MyEventHandler Handler;`	

explicit	Used as a modifier for user-defined class operators converting the parameter type to this type. Similar to C++'s constructor accepting parameter type. Conversions with the *explicit* keyword imply that a client must explicitly use a cast operator for it to be called. Server code that defines the operator should use *explicit* if the conversion may cause an Exception or information loss	`public class MyType`
		`{`
		
		`public static` **`explicit`** `operator MyType(int i)`
		
		`{`
		
		`//write code`
		
		`///converting int to`
		
		`//MyType`
		
		`}`
		
		`}`
			`public class MyClass`
			`{`
			
			`public MyClass(int i)`
			
			`{`
			
			`//write code to convert`
			
			`//this holding i`
			
			`}`
			
			`}`

extern	Used as a modifier in an empty method definition, with the implementation usually existing in an external dll file. Similar to C++.	`[DllImport("User32.dll")]` `public static` **`extern`** `int MessageBox(int h, string m, string c, int type);`	

fixed	Must be used in "unsafe" mode for manipulating pointers (pointers are allowed in C# but should be used sparingly).	`int[] ia = {1,2,3};`
		`fixed` `(int* i = &ia)`
		
		`{`
		
		`}`	

foreach	Defines a looping statement in C# for collections implementing specific enumeration interfaces. Very nice language feature used when every	`using System.Collections;`
		`ArrayList list = new ArrayList();`
			`Vector v = new Vector();`
			`v.addElement (new Integer(1));`
			
			`v.addElement(new Integer(2));`

	element in an enumeration will be inspected. Any necessary casting is done implicitly for the developer in case of generic container use. Compare to an equivalent Java code segment, which requires the developer to explicitly cast during inspection.	`list.Add(1);`
		
		`list.Add(2);`
		
		`foreach (int i in list)`
		
		`{`
		
		`int j = i;`
		
		`}`
			`for (int i = 0; i < v.size(); i++)`
			
			`{`
			
			`int j = (Integer)v.elementAt(i).toInt();`
			
			`}`

get*	Not truly a keyword (not reserved). Can be used as an identifier, but avoid. If used as `get { }` then defines a class accessor function. Very nice from the client's perspective, because it appears as if he is directly accessing some data in the class when he is not. Nice for the class writer because he can perform other functionality before returning data.	`class MyClass`
		`{`
		
		`private int m_int;`
		
		`public int MyInt`
		
		`{`
		
		`get`
		
		`{`
		
		`return m_int;`
		
		`}`
		
		`}`
		
		`MyClass m = new MyClass();`
		
		`Int m = m.MyInt;`
			`class MyClass`
			`{`
			
			`private int m_int;`
			
			`public int getInt()`
			
			`{`
			
			`return (m_int);`
			
			`}`
			
			`}`
			
			`MyClass m = new MyClass();`
			
			`Int m = m.getInt();`

implicit	Similar to the *explicit* keyword defined above, but implies that a developer does not have to use an explicit cast for conversion. Converts the class to the	`class MyType`
		`{`
		
		`public static implicit operator int (MyType m)`
			`class MyType`
			`{`
			
			`public int getInt()`

	parameter type. Similar to C++'s conversion operator.	`{`
		`//code to convert this to`
		`//int`
		`}`
		`}`
			`{`
			`//write code to`
			`//convert`
			`//this to an int`
			`}`
			`}`

in	Keyword prefix operator used in a *foreach* loop, described above. Provides readability and a signal to the compiler that the container will be to its right.	See `foreach` example.	

new*	The *new* keyword has a context-sensitive meaning in C#. While it is used as an operator that returns a reference to a newly created object in both languages, it is also used in C# as a modifier to hide previously defined methods, properties, and indexers, for example, in a base class with the same signature or name. Please read the documentation for more information.	`public class MyClassBase`
		`{`
		`public virtual void foo()`
		`{`
		`}`
		`}`
		`public class MyClass : MyClassBase`
		`{`
		`public` **`new`** `void foo()`
		`{`
		`//hides base version`
		`}`
		`}`
			`public class MyClassBase`
			`{`
			`public void foo()`
			`{`
			`}`
			`}`
			`public class MyClass extends MyClassBase`
			`{`
			`//must create actually`
			`//new method signature`
			`public void foo2()`
			`{`
			`}`

			`}`
object	Based on the object data type, used for boxing. (Note: I am not completely aware of all of the nuances between using this keyword and object at the time during writing this document. Please read Microsoft's online documentation.)	`object o = 1;`	

operator	Keyword used in a class method overloading a supported operator. Operator overloading is not supported in Java.	`public class Vector3D`
		`{`
		
		`public static Vector3D operator + (Vector3D v)`
		
		`{`
		
		`return (new Vector3D(x+v.x,y+v.y,z+v.z));`
		
		`}`
		
		`}`
			`public class Vector3D`
			`{`
			
			`public Vector3D add(Vector3d two)`
			
			`{`
			
			`//add implementation`
			
			`}`
			
			`}`

out	Method parameter and caller modifier that signals that the parameter may be modified before return. Should be used sparingly.	`public class MyClass`
		`{`
		
		`public int sort(int[] ia,out int)`
		
		`{`
		
		`//add implementation`
		
		`}`
		
		`}`
		
		`int[] ia = {1,7,6};`	

		int i; int s = MyClass.sort(ia,**out** i);	

override	Method or property modifier in C# that implies that this method should be called instead of the super class's virtual method in case a more generic reference is held at run-time.	public class A
		{
		
		public virtual int Test()
		
		{
		
		return 0;
		
		}
		
		}
		
		public class B : A
		
		{
		
		public **override** int Test()
		
		{
		
		return 1;
		
		}
		
		}
		
		A a = new B();
		
		int I = a.Test(); //1 is returned
			public class A
			{
			
			public int Test()
			
			{
			
			return 0;
			
			}
			
			}
			
			public class B extends A
			
			{
			
			public int Test()
			
			{
			
			return 1;
			
			}
			
			}
			
			A a = new B();
			
			int I = a.Test();
			
			//1 is returned. All methods
			
			//in Java are virtual

params	Method formal parameter modifier that allows a client to pass as many parameters to the method as he wants.	public class MyClass {	public class MyClass {

	Nice language addition similar to the . . . in C++.	`public static void Params(`**`params`**` int[] list)`
		
		`{`
		
		`//add implementation`
		
		`}`
		
		`}`
		
		`MyClass.Params(1,3,7);`
			`public static void ParamSimulate(int[] ia)`
			
			`{`
			
			`}`
			
			`}`
			
			`int[] ia = {1,3,7};`
			
			`MyClass.ParamSimulate(ia);`

ref	Similar to *out* parameter above, except a *ref* parameter is more like an "in/out" param: it must be initialized before the call, where it is not required to initialize an "out" parameter before the method call.	See `out` example, but use **`ref`**.	
sbyte	A signed byte between -128 to 127	`//define an sbyte and assign` **`sbyte`**` mySbyte = 127;`	

set*	Please see *get* above. Also not a keyword, but treat it like it is. Allows a client to set data on a class instance.	`public class MyClass`
		`{`
		
		`private int m_int;`
		
		`public int MyInt`
		
		`{`
		
		`set`
		
		`{`
		
		`m_int = `**`value;`**
		
		`//see value below`
		
		`}`
			`public class MyClass`
			`{`
			
			`private int m_int;`
			
			`public void set(int i)`
			
			`{`
			
			`m_int = I;`
			
			`}`
			
			`}`
			
			`MyClass m = new MyClass();`
			
			`m.set(3);`

		`}`
		
		`}`
		
		`MyClass m = new MyClass();`
		
		`m.MyInt = 3;`	

sizeof	Prefix operator similar to C++, accepting an expression as an r-value. Should be used sparingly, as it is only supported in unsafe mode.	`int i = 3;` `int s = sizeof(i);`	

stacalloc	Used for allocating a block of memory on the stack. Should be used sparingly, as it is only supported in unsafe mode.	`public static unsafe void Main()`
		`{`
		
		`int* fib = stackalloc int[100];`
		
		`int* p = fib;`
		
		`*p++ = *p++ = 1;`
		
		`for (int i=2; i<100; ++i, ++p)`
		
		`*p = p[-1] + p[-2];`
		
		`for (int i=0; i<10; ++i)`
		
		`Console.WriteLine (fib[i]);`
		
		`}`	

string	Alias for the System.String class.	`string s = new String();`	`String s = new String();`

struct	Similar to a struct in C++. Lightweight, where a constructor is only called if *new* is used to create.	`struct MyStruct`
		`{`
		
		`int MyInt;`
		
		`}`
			`class MyStructSimulate`
			`{`
			
			`private int m_int;`
			
			`public int get()`

			`{`
			`return (m_int);`
			`}`
			`}`

this*	Context sensitive. C# allows for indexers, while Java does not. But *this* in both also returns a reference to the actual class member.	`class MyClass`
		`{`
		`private int[] m_array;`
		`public int **this**[int index]`
		`{`
		`get`
		`{`
		`return (m_array[index]);`
		`}`
		`set`
		`{`
		`m_array[index] = value;`
		`}`
		`}`
		`}`	

typeof	Prefix operator accepting an expression as an r-value returning the runtime type of the object.	`MyClass m = new MyClass();`
		`Type t = **typeof**(m);`
		`//same as m.GetType();`
			`MyClass m = new MyClass();`
			`Class c = m.getClass();`

uint	Unsigned integer.	`uint i = 25;`	
ulong	Unsigned long.	`ulong l = 125;`	

unchecked	Opposite of "checked" above. No arithmetic exceptions must be caught in the block. This is the default behavior.	`Unchecked`
		`{`
		
		`}`	

unsafe	Defines an "unsafe" block of code in C#. Should be used sparingly.	`public static unsafe unsafeMethod();`	
ushort	Unsigned short.	`ushort u = 7;`	

using*	Context sensitive in C#. Defines a block on an expression, where it is guaranteed that dispose is called on the expression after the block is at the end.	`MyClass m = new MyClass();`
		`using (m)`
		
		`{`
		
		`}`	

value*	Proxy for a passed value into a set function. Please see *set* and *get* above.	`public class MyClass`
		`{`
		
		`private int m_int;`
		
		`public int MyInt`
		
		`{`
		
		`set`
		
		`{`
		
		`m_int = value;`
		
		`}`
		
		`}`
		
		`}`	

virtual	Method or property modifier in C#. Similar to C++. All Java methods are virtual by default, so Java does not use this keyword.	`public class MyClassBase`
		`{`
		
		`public virtual int GetInt()`
		
		`{`
		
		`return 3;`
		
		`}`
		
		`}`
			`public class MyClassBase`
			`{`
			
			`//all methods virtual by`
			
			`//default.`
			
			`public int GetInt()`
			
			`{`
			
			`return 3;`
			
			`}`
			
			`}`

Supported in Java but Not in C#

The following keywords are supported in Java but not in C#.

Java Keyword	Notes	Java Example	C# Equivalent
native	Since Java was designed to run on any supported operating system, this keyword allows for interoperability and importing code compiled in some other language.		
transient	Supposedly currently unused in Java.		

synchronized*	Context-sensitive keyword. When used as an instance method modifier, guarantees that the single instance mutex will be gained at function entrance and released just before the function exits. If used as a static method modifier, then the class mutex will be used instead. Also allowed as a	`public synchronized void LockAndRelease()`
		`{`
		
		`//instance lock`
		
		`//implicitly`
			`public void LockAndRelease()`
			`{`
			
			`//lock must`
			
			`//be`
			
			`//called`

	class modified, which means that all class access is synchronized implicitly.	`//acquired`
		
		`//write code here`
		
		`//instance lock`
		
		`//implicitly`
		
		`//released`
		
		`}`
			`//explicitly`
			
			`lock(this)`
			
			`{`
			
			`//code`
			
			`//here`
			
			`}`
			
			`}`

throws*	Slightly different meaning in C# and Java. The exception must be caught by the client in Java if it is not a RuntimeException.	`public void foo() throws MethodNotFoundException`
		`{`
		
		`}`
			`public void foo()`
			`{`
			
			`}`

Keyword Analysis

Simply by looking at the above tables and counting the number of keywords reserved in each language, you may be tempted to say, "Game over, dude! C# rocks! Let's port our Java code to C#, then snag a six pack of snow cones and check out *Jackass the Movie* on the big screen!" It may be wise to think before you do. Have one fat free yogurt instead—it has to be healthier for you. And maybe see *Life as a House* at home on DVD. I highly recommend it. After all, quality is often more important that quantity. But more important, if you really feel the need to jump, make sure that you look before you leap—while watching *Jackass* is painful enough, you surely don't want to risk making an appearance in the sequel. [2]

Returning to reality and the original argument, C# can reserve as many keywords as it wants, but if no one uses them, it doesn't matter. And more keywords in a language necessarily implies that the set of possible identifiers at a developer's disposal decreases (albeit by a very tiny number compared to what's possible). But there also is the danger that a language does not reserve a keyword that it should. For example, *virtual* is not reserved in Java. But what if Java wishes to *extend itself* in the future by defining *virtual*? It can't. It may break code written before the inclusion, which would ironically make code originally built to run on any operating system now run on none. There is nothing wrong with reserving an unused keyword and documenting that it currently has no meaning. But the explicit set of keywords that a language reserves tells very little in itself. What's really important? What's the implied meaning and context in which these keywords are used? Do they make the developer's job any easier? And they really *only* make the developer's job easier. After all, all programming languages are equivalent. There is nothing that can be done in C++ that can't be done in C# that can't be done in Java that can't be done using assembly language that can't be done using 1's and 0's by a good typist with an incredible memory. With this in mind, there is a set of keywords above that really stand out. Let's start out by discussing a smooth operator. . . .

Operator

The first, and maybe most important, is *operator*. This keyword allows operator overloading in C#, something that is not supported in Java. Naturally, operator overloading is not necessary, because a developer can always create a standard method that accepts parameters to perform the same function. But there are cases when applying mathematical operators is completely natural and therefore highly desirable. For example, say that you have created a `Vector` class for graphics calculations. In C#, you can do the following:

```
public class Vector
{
   //private data members.
   private double m_x;
   private double m_y;
   private double m_z;

   //public properties
   public double x
   {
      get
      {
         return (m_x);
      }
      set
      {
         m_x = value;
      }
   }

   //define y and z Properties here
   . . .

   public Vector()
   {
      m_x = m_y = m_z = 0;
   }

   public Vector(double x, double y, double z)
   {
      m_x = x;
      m_y = y;
      m_z = z;
   }

   public static Vector operator + (Vector v2)
```

```
    {
       return (new Vector(x+v2.x,y+v2.y,z+v2.z));
    }

    //define -, *, whatever you want here. . .
 }
```

The C# client using this class can then do something like the following:

```
 Vector v = new Vector();  //created at the origin
 Vector v2 = new Vector(1,2,3);
 Vector v3 = v + v2;  //sha-weet. . .
```

In Java, the following could be done:

```
 public class Vector
 {
    private double m_x;
    private double m_y;
    private double m_z;

    public Vector()
    {
       m_x = m_y = m_z = 0;
    }

    public Vector(double x, double y, double z)
    {
       m_x = x;
       m_y = y;
       m_z = z;
    }

    public double getX()
    {
```

```
        return (m_x);
    }

    //define accessors for y and z below
    . . .

    public Vector addTwoVectorsAndReturnTheResult(Vector v)
    {
        return (new Vector(m_x+v.x,m_y+v.y,m_z+v.z));
    }
}
```

Then the Java client would have to do something like:

```
Vector v = new Vector();
Vector v2 = new Vector(1,2,3);
Vector v3 = v.addTwoVectorsAndReturnTheResult(v2);
//You killed operator overloading!
```

What can you say about the C# code above? Well, if you appreciate my coding style and operator overloading: "Sha-weet. . . . ." What can you say about the Java code above? If you like or dislike my coding style, it's most likely, "You killed operator overloading!" (Maybe worse. But this isn't some television show aimed at kids, so we can't use really foul language here.) And you might still swear uncontrollably even if I had used some reasonable method name such as *add* in the Java's version—the long method name was only used for effect. While the C# code is completely natural to developers who have taken some math, the Java code is completely unnatural. While the intent of the C# client is immediately obvious, the Java client code must be inspected before the light bulb turns on. Some people may argue that operator overloading is not really important, and so it is not a "biggy" that Java does not allow it. If this is so, why does Java allow the '+' operator to be used on the built-in java.lang.String class, then? Why is the String class more important that any class that you or I build? So string operations are common, you may argue. But just by this example Java shows that it feels that operator overloading can be a good thing in some situations. I guess these situations only exist where Java has the control.

Why did Java omit operator overloading? I don't know. One thing that I do believe: it was a mistake. Operator overloading is very important because it allows developers to write code that seems natural to them. It could be and has been argued ad nauseam that "operator overloading can be easily overused, so it shouldn't be allowed." That would be like saying that "cars are in accidents so let's make people walk." People can and do make mistakes but it would be a bigger mistake to make them walk 20 miles to work every day. If you can't trust programmers to make good decisions, then their code won't work anyway, even without operator overloading. Give me back my car—I get enough exercise when I run that stupid treadmill. And give me back my operator overloading—I get enough typing practice when I write these "smart" papers.

Delegate

A *delegate* in C# is like a function pointer in C++. They are both used in situations where some function should be called, but it is unclear which function on which class should be called until run time. While both of these languages require methods to follow a pre-defined signature, each allows the name of the individual function to be anything that is legal as defined by its respective language.

One nice feature of both C++ function pointers and C# delegates is how they both handle *virtual* functions. In C#, if you create a base class that implements a *virtual* method with a signature matching a *delegate*, then subclass this base and *override* the method, the overriding method will be called on a *delegate* call if a base reference actually holds an instance of the subclass. C++ actually has similar behavior, although its syntax is more awkward.

What is the motivation for delegates in C#? One place they come in handy is for event creation and handling. When something happens during program execution, there are at least two ways for a thread to determine that it has happened. One is polling, where a thread simply loops, and during every loop block, gains some lock on data, tests that data for the "happening," releases the data then sleeps for a while. This is generally not a very good solution because it burns CPU cycles in an inefficient manner since most of the tests on the data will return negative. Another approach is to use a publisher-subscriber model, where an event listener registers for some event with an event broadcaster, and when something happens, the broadcaster "fires" an event to all listeners of the event. This latter method is generally better, because the logic is simpler, particularly for the listener code, and it's more efficient, because the listener code runs only when an event actually occurs.

Java uses this model to handle events, particularly but not limited to classes associated with GUIs. A listener implements some well-defined interface defined by the broadcaster, then registers with this broadcaster for callback in case an event of interest occurs. An example would be a java.awt.event.ItemListener, which can register for item changed events in a java.awt.Choice box. Another would be a MouseListener, which can listen for such events on a generic java.awt.Component such as mouseEntered or mousePressed. When an event occurs, the broadcaster then calls the pre-defined interface method on each registered listener, and the listener can then do anything it wants.

In C#, an *event* and a *delegate* are defined and then implemented by some class or classes, so that an *event* can be broadcast to anyone registering for this *event*. Most if not all C# Controls already have logic to fire many different predefined events to registered listeners, and all you have to do is create a Control subclass instance, create a "listener" class that implements a *delegate*, then add that listener to the Control's *event* list. But C# even allows you to create your own events and event handlers by using the following procedure: [3]

1. Define and implement a subclass of *System.EventArgs* with any pertinent properties.
2. Define the *delegate* function prototype of the form `public delegate void <delegateMethodName>(object sender,EventArgs e)` at the namespace scope.
3. Define a class that internally defines an *event* of the form `public event <delegateMethodName> <eventListName>` and implement code that fires this event to its listeners when appropriate, by passing *this* and a new instance of the `EventArgs` subclass.
4. Define at least one class that implements the *delegate* method as a listener.
5. Create at least one listener and one broadcaster instance, and add the listener to the broadcaster's *event* list.

It should be noted that, in some cases, the listener implementation code may want to spawn another thread so that other listeners may also receive this *event* in a timely fashion in case this code performs any lengthy processing. It should also be possible for the class firing the event to spawn a thread. But in these scenarios, you must be careful to use synchronization techniques on both the `EventArgs` data and the object sender, since multiple threads may attempt to access these simultaneously. Just running some simple tests, it appears as if the *delegate* implementers are called in a queue-like fashion in which they were added, with the same parameter references, so take this into account.

There are some differences in implementation between C# and Java in regards to creating, firing and receiving events, but overall, they are very similar since they both use a publisher-subscriber model. However, there are some very subtle differences, particularly in client implementation, that suggest some advantages and disadvantages in

the approaches.

One problem with the Java approach is that, while a single listener can register for events on multiple like-Components, the same method will be called on the listener regardless of which individual Component actually triggered the event. This happens because the broadcaster references this listener as a well-defined interface, and only one method exists for each event type on that interface. So, if the same listener registers for events on more than one like Component, it must have some nasty if-then [else] (ITE) statement inside the event handler method to first determine which Component triggered the event in case the action to perform is Component-dependent, which is usually the case. In C#, however, a class that registers for some event can create one specific handler method for each Component that may trigger this event. Why can it do this? Because methods implementing a *delegate* must follow the delegate's exact signature *except* it may use any method name that it wishes. So C# avoids this problem.

```
//Java Code
public class MyButtonListener extends Window implements ActionListener
{
   //two Buttons for accepting or rejecting some question.
   private Button m_acceptButton;
   private Button m_rejectButton;

   /**
    * For now just creates two Buttons and adds this as a listener
   */
   public MyButtonListener()
   {
      m_acceptButton = new Button("OK");
      m_acceptButton.AddActionListener(this);
      m_rejectButton = new Button("Cancel");
      m_rejectButton.AddActionListener(this);
      //write code to add these Buttons to me and for layout
   }

   //ActionListener events
   /**
    * Called when a Button is pushed
    */
   public void actionPerformed(ActionEvent e)
   {
      if (e.getSource() == m_rejectButton)
      {
         //write rejection code
      }
      else if (e.getSource() == m_acceptButton)
```

```
      {
         //write acceptance code
      }
   }
   //End ActionListener events
   //implement any other necessary code here
}
```

This may not seem like a big deal, but it does involve some bookkeeping, and it really isn't an object-oriented approach once you enter the event handling code—the Java developer above has become an "if-then [else] programmer," which is rarely good. In comparison, here is the same code in C#:

```
//C# code
public class MyButtonListener : Form
{
   public MyButtonListener()
   {
      Button accept = new Button();
      accept.Text = "OK";
      accept.Click += new EventHandler(AcceptClick);
      Button reject = new Button();
      reject.Text = "Cancel";
      reject.Click += new EventHandler(RejectClick);
      //write code here to add and layout the Buttons
   }

   //Event handlers for our buttons
   private void AcceptClick(object sender, EventArgs e)
   {
      //only called if accept Button clicked
   }

   private void RejectClick(object sender, EventArgs e)
   {
      //only called if reject Button clicked
   }
   //end event handlers.
}
```

Note some items above: first, the C# code in this case automatically knows implicitly which *System.Windows.Forms.Button* triggered the event based upon which method was called, so it doesn't need to perform an additional test. Second, the C# code isn't required to hold references to the Buttons because it doesn't have to make comparisons in the event handler methods. To be completely fair, the Java code doesn't actually have to either, but it would then have to either set some Action on each java.awt.Button and get that information back out in its single handler method when called, which isn't too bad—it isn't perfect however since a test must still be performed; or it would have to inspect the label of the Button for the comparison, which isn't too good—it makes internationalization potentially impossible later. Another possible approach is to create a listener instance for each Component, but this naturally has the disadvantage of requiring more memory, and it still doesn't always solve this problem. Another problem may occur if you misspell the label on the Button, remember to fix this visual error later on the Button but forget to change the handler logic code at the same time. There are other possible solutions but none of them are very satisfying. Third, the *delegate* functions in C# may be declared as *private*, which is very nice. If this class instance is referenced by another for some reason independent of event handling, then the latter class will not be able to call these handler methods directly on the former, which is good. In Java, all interface methods must be *public*, so any class that implements an interface will not only expose these functions to the broadcasters but to everyone else, which is bad.

It should be noted that the C# code above could also have defined and implemented one method to handle clicks on both Buttons, simulating the Java approach. But in this case it knows that it will only have two Buttons, and the action performed will be presumably Button-dependent, so it made the correct choice. So C# is a just little more flexible and discourages "if-then [else]" programming.

One more item to consider about the above code: You will notice that Components have been added and modified by hand. Visual Studio .NET has the Form Designer, which sometimes makes building GUIs more simple for the developer by allowing him to drag child Components to a specific location on a parent Form, set properties in a separate panel, and create event handler stubs in a visual manner. While these tools are great, as they automatically write code for you behind-the-scenes, it is my belief that it is still best to build GUIs manually. Why? When you use drag-and-drop, it implies that you already know what your user interface will look like, which may not be the case and rarely is. Often, a form must change drastically at run time based on a single user action, and you can't build all possible combinations in the Form Designer beforehand, unless you have a thousand years or so.

Just look at a pretty decent user interface, Internet Explorer. While it may know what types of Components that it may draw at compile time (although it only really has to have knowledge about some possible base interface, then create a Component instance by name and reference the interface, which would be better), it has no idea what specific Components it will create and load and how they will be laid out until a Web site is visited by the user. I would assume that the IE code is very dynamic, and most of the code is therefore written by hand. Paradoxically, you may often find that the amount of code in your application may actually be less when you build user interfaces by hand rather than using a designer, particularly if you recognize and take advantage of patterns learned while building user interfaces. This is because you are smarter than the designer, as you write generic, reusable code; the designer does a good job of writing correct, but very specific code to solve a single problem. So maintenance should be easier, too, if your design is good.

Both Java and C# are very nice when it comes to basic GUI patterns: create a Component, set its properties, create and add event handlers for the new Component, define some layout scheme, and add it to its parent Component. This often suggests a very recursive and natural solution. While you can do similar things in languages such as C++ using Visual Studio and MFC, for example, it is not as easy for a number of reasons. In C# and Java, it is fairly straightforward. It is therefore my recommendation that you use tools such as the Form Designer when first learning a new language, graphics package or IDE because you can drag and drop, set some properties and create event handlers, then inspect specific code "written" by the designer and observe noticeable patterns for your development. Treat the Form Designer like your own personal trainer: use him then lose him. Oh, but don't forget to thank him after he teaches you how to write lean and mean code.

One minor advantage of Java's approach to event handling is that it seems to be simpler to learn than C#'s version, maybe because most Java developers are already familiar with interfaces, and event handling is just built on top of them. And Java's approach seems to be somewhat more "elegant" from a purist's perspective. In C# you

must learn first how to use delegates and events, so there are a few more "hoops" to jump through. But once you learn how each works, definition and implementation is done in both with approximately the same ease.

C# appears to be superior in these cases. First, it allows delegates, which are not supported in Java, and thus allows developers to make dynamic method calls on any class without using reflection at run time. Even though the use of delegates should be held to a minimum, support exists just in case they are needed. Second, event handling built on top of delegates and events appears to also be superior to Java's method of using interfaces because of the above pragmatic issues.

get/set/value (Properties)

These are not reserved keywords in C#, although they probably should be. A developer can use these as identifiers, but it is best to avoid doing so just in case C# decides to reserve them in the future. They must be discussed together, because they all are used to define standard user-defined property accessor functions in C#.

C# is not the first language to allow class properties. For example, ATL/COM allows you to create accessor functions on interface definitions using class implementations. The following example demonstrates how these are defined in ATL.

```
//idl file
[
   object,
   uuidof(. . .),
   dual,
   helpstring("IMyClass interface"),
   pointer_default(unique)
]
interface IMyClass : IDispatch
{
   [propget, id(1), helpstring("property Number")]
    HRESULT Number([out, retval] long *pVal);
   [propput, id(1), helpstring("property Number")]
    HRESULT Number([in] long newVal)];
};

//header file
class ATL_NO_VTABLE CMyClass:
   public CComObjectRootEx<CComSingleThreadModel>,
   public CComCoClass<CMyClass, &CLSID_MyClass>,
   public ISupportErrorInfo,
   public IDispatchImpl<IMyClass, &IID_IMyClass, &LIBID_MYCLASSATLLib>
{
public:
   CMyClass() : m_Long(0)
   {
```

```
   }

DECLARE_REGISTRY_RESOURCEID(IDR_MYCLASS)

DECLARE_PROTECT_FINAL_CONSTRUCT()

BEGIN_COM_MAP(CTicTacToeBoard)
   COM_INTERFACE_ENTRY(IMyClass)
   COM_INTERFACE_ENTRY(IDispatch)
   COM_INTERFACE_ENTRY(ISupportErrorInfo)
END_COM_MAP()

// ISupportsErrorInfo
   STDMETHOD(InterfaceSupportsErrorInfo)(REFIID riid);
public:
   STDMETHOD(get_Number)(/*[out, retval]*/  long *pVal);
   STDMETHOD(put_Number)(/*[in]*/ long newVal);

private:
   long m_Long;
}

//implementation file
STDMETHODIMP CMyClass::get_Number(long *pVal)
{
   *pVal = m_Long;
   return S_OK;
}

STDMETHODIMP CMyClass::put_Number(long newVal)
{
   m_Long = newVal;
   return S_OK;
}
```

The ATL code for my COM object is more long-winded than this paper. Luckily, the client code using smart pointers is easier:

```
IMyClassPtr myClass(__uuidof(MyClass));
myClass->Number = 3;
```

```
long l = myClass->Number;  //l gets 3
```

(Grimes, pp 84-89. His Easter example was used as a template for this COM example.)

As you can see, ATL code can be quite complex. To be fair, the ATL class wizards in any version of Visual Studio will do the majority of work for you under-the-covers by creating and modifying files such as definition (.idl), header (.h), implementation (.cpp), and registry scripts (.rgs), where these files will be nearly complete minus most implementation code (even these are stubbed for you). But attempting anything beyond the basics will require hand-modifying these files, which can be somewhat tricky, and requires near-expert knowledge of the inner details of COM. In contrast, the following equivalent example demonstrates how simple and elegant both C# class implementation and client code can be:

```
public class MyClass
{
   //private data member
   private long m_long;

   //public property
   public long Number
   {
      get
      {
         return (m_long);
      }
      set
      {
         m_long = value;
      }
   }

   //constructor
   public MyClass()
   {
      m_long = 0;
   }
}
```

The client code could look like the following:

```
MyClass m = new MyClass();
m.Number = 3;  //m_int gets 3
long m = m.Number;  //m gets 3
```

Which one is easier? I'll let you make the call. I've made my choice.

So why are properties important? They allow a developer to access private data indirectly in a natural way. Why is it important to access this data indirectly? It allows the developer of the class which exposes these properties to perform other "bookkeeping," such as locking and unlocking the private data, make other function calls, and so on, during the *get* and *set* functions, and hide this functionality from the client code. And it is possible to declare a property as *virtual*, so that a subclass can *override* the implementation of this property if it so chooses. Another nice feature of properties: if a developer uses Reflection, he can access these properties more easily in a generic way. It is possible to use non-standard *set* and *get* accessor functions, but the syntax of these functions must be well-defined and negotiated beforehand for this to work without built-in language support. Some language extensions have simulated accessor functions by instructing classes to use the forms get_<property> and set_<property>, which implies that properties are important, since support is simulated after a language definition is defined.

Are properties necessary? No. Java doesn't supply this functionality. (Although J# [4] does, using the above simulation method. Even managed C++ does, too.) A developer can always use non-standard get and set accessor methods for variables. But it supplies a standard way for client and server code to interact and pass messages.

Enum

User-defined enumeration types are not supported in Java. A technical explanation: Yikes. 'Nuff said.

OK; maybe not enough said here. But this one irritates me a little bit. While an enum *should be used sparingly, because it doesn't really imply an object-oriented approach, there are situations where they make sense to use. They are very nice to use in situations where you don't want to create an entirely new class to describe data, particularly hidden inside a class where only a few options exist. While I would agree that they should be used sparingly, a language doesn't seem complete without them. In the words of an entertaining and controversial talk show host, "What say you, Java?"*

This

C# allows indexers on classes, while Java does not. "This" is a very nice feature, when a class is designed to hold some enumeration of objects. You may only define one indexer on any class, but this is actually a good thing to avoid ambiguity. So, if your class will hold several different lists of objects, then it may be best to not use this functionality. If it holds only one, then it makes sense to use indexers.

There are some interfaces and functionality that a class needs to implement to support *this* functionality, and I recommend that you read the online documentation about *indexers*. If you implement the correct interfaces on your class using this information, the client of your collection may apply the *foreach* statement on it, which is very nice.

Struct

Structs are allowed in C# but not Java. While this is not really a big deal (just use a *class*), there are some good things about C# structs, including being more efficient in some situations. I recommend reading about the differences between a *struct* and a *class* in the online documentation. I also highly recommend using a *class* instead in most circumstances. Think twice before committing your data to a *struct*. Then think about it again.

out/ref

These keywords allow a callee control over side-effecting the reference to formal parameters, so that the caller will see the change after the call is made. This is different than simply side-effecting internal data that the reference holds; it also allows side-effecting what the calling reference actually refers to.

Examining the *out* keyword first, it is very similar to using a non-const "pointer to a pointer" in a C++ function call. When the method is called, the address of the original pointer is side-effected, so that it points at new data. Where would this be useful? Let's say that you are coding in C++, and you need to create a function which accepts an array, returns the index of the largest element and sets a passed-in pointer to access the actual element in the array that is the largest, just in case the developer wishes to modify this array element later. Maybe not the smartest or the safest thing to do, but hey, it's C++: we can do whatever we want.

```
//PRE:  ia and size initialized, and element either NULL or uninitialized
//POST: index or largest element returned if possible else -1, and element points to largest element
//      else NULL
//ia - the array to inspect
//size - the size of array ia
//element - after the call references the actual largest element in the array else NULL if empty
//return - zeroth relative index of the largest element if array size > 0 else -1
int Largest(int* ia,int size,int** element)
{
   //if array is empty then no largest element
   if (size < 1)
   {
      (*element) = NULL;
      return (-1);
   }

   //has one element.  Default to zeroth element
   int nBiggest = (*ia);
   int nIndex   = 0;
   (*element)   = ia;
   int nCount   = 1;

   //start the search at element one.  We've already
   //seen and accounted for element zero.
   for (int* i = ia+1; nCount < size; i++)
   {
      if ((*i) > nBiggest)
      {
         nBiggest   = (*i);
         nIndex     = nCount;
```

```
            (*element) = i;
         }
         //always increment for the next element.
         nCount++;
      }

      //return the index number of the largest element
      return nIndex;
   }
```

The C++ client code could then look like:

```
int ia[] = {1,3,2,7,6};
int* i = NULL;
int nBig = Largest(ia,5,&i);  //nBig should be 3, and i should reference the '7' element
(*i) = 6;  //now i points to an int with the value 6, and the array's third element, zero-relative, //is 6
```

Sample C# prototype and client code could look like this:

```
public class MyInt
{
   private int m_int;

   public int Value
   {
      get;set;
   {

   public MyInt();

   public static int Largest(MyInt[] ia,out MyInt element);
}

MyInt[] ia = {1,3,2,7,6);  //pseudo
MyInt i;
```

```
int nBig = MyInt.Largest(ia,out i);  //nBig should be 3, and I should reference the '7' element
i.Value = 6;  //now i's m_int value is 6, and the array's third-element, zero-relative, is 6
```

It should be noted that an *int* wrapper object was needed to simulate the behavior of the C++ code using C#. This is due to *boxing*, which implicitly converts value types to objects and vice versa in C#. While using an *int* does mostly work in this scenario by returning the correct index and element values, the actual element in the array cannot later be side-effected by changing `i` later if it is of type *int* because it would then be a *value* type, not an object or an actual reference to the element in the array. Using the wrapper here works the same as the C++ version because this wrapper is an object. So, while boxing is generally a "good thing" in most scenarios, a developer should beware of some possibly unexpected but as-designed C# behavior in this situation.

Another item of interest: Notice that the array passed into the C# `Largest` version does not need a corresponding `size` parameter, because any array can be queried for its size through the *Length* property. Very nice, and very safe. It is very common to see C++ APIs littered with additional array size parameters that tend to clutter code. You will not see this in C# and Java.

Returning to the new keyword modifiers supported in C#, if the caller and the callee add the *out* modifier to the parameter, then the actual object that the reference refers to may change during the function call. This works on primitives and objects in the same manner, with some slight differences with value types, as discussed above.

The *ref* keyword has a very similar meaning to the *out* keyword. The only difference: the *out* parameter does not have to be initialized before the call is made, while the *ref* parameter does. Basically, a *ref* parameter implies that the method can safely inspect the parameter before modifying it in any way.

I still believe in most circumstances that it is best to create a composite object if multiple items must be returned from a function call. But there are some situations where using out *or* ref *would really come in handy.*

I found out that this functionality does not exist in Java, much to my dismay, when I was building user interfaces. I had a situation where I needed to create an object instance dynamically at runtime using reflection and set another data's field to this new object also dynamically, which meant that I had to also pass the parent object that held this field. If I were coding in C#, I can imagine that I could have just used an out parameter on a method call, and I would have then side-effected the actual parent object reference by setting it to the newly created object, so I wouldn't have needed to also pass this parent data. Bummer.

foreach/in

This defines a new looping statement in C#. This is supported in C# but not Java. The nicest thing about a *foreach* statement: it implicitly handles casting for the user during container iteration. While this statement is not necessary (use a *for* loop instead, for example) it simplifies stepping through an enumeration when all items need to be inspected.

You may create objects yourself which will allow this statement to iterate through your object's enumeration. I recommend looking at this documentation, which will tell you what interfaces to implement and what to do to make this happen.

virtual/override/new

These three keywords go hand-in-hand in C#, allowing classes and their subclasses more control over static and dynamic binding. In Java, all methods are *virtual* by default and can only be made non-virtual by using the *final* keyword. In C#, all methods are non-virtual by default, and can only be made *virtual* through the same keyword. Since these languages are diametrically opposed on this issue, it seems to beg the question: Should methods be implicitly *virtual*, implying that Java got it right;

or should they be implicitly non-virtual, which means that C# got it right? This question is truly subjective, almost philosophical, and there are advantages and disadvantages to both solutions.

One disadvantage with C# is that the class designer has to guess which methods should be *virtual* and which methods should not, and if he guesses wrong, then problems ensue (Lippman, p 530). For example, what if a base class is built which is intended for derivation, where some but not all of the methods are declared as *virtual*? Then, another developer creates a subclass of this *base* class, and determines that he needs to override a non-virtual method for some reason—maybe there's a bug in the original implementation? Maybe he needs to perform some other function before the *base* class' method should be called? And so on. He simply can't override the base's functionality because it is non-virtual. He can create a function with the same signature, but this method will only be called if the compiler can determine the exact type of the object at compile time. This defeats the idea of inheritance and *virtual* functions anyway, since code is usually written so that the most abstract reference possible holds a derived instance so the caller doesn't know and doesn't care what version of the function will be called. So the "overriding" method won't be called in this scenario. The only workaround may be to redefine the *base* class method by adding the *virtual* modifier and then recompiling, which is not necessarily pretty or possible. For example, what if you are using a third-party library? Try calling a vendor at 3 AM and requesting a change.

Another C# disadvantage is that the class designer and developers inheriting from this class must do more work. In Java, the class designer creates a class, and assumes that any method not declared as *final* may be overridden by any subclass. In C#, the class designer must choose which methods to declare *virtual*, which requires more brainpower. Both he and the developers who subclass this *base* are also required to do more typing and thinking.

One advantage to C# is that the code should run faster, because there should be fewer *virtual* methods. The compiler will more often determine at compile-time which actual functions should be called. In Java, methods must almost always be treated as virtual so these decisions must be put off until run time.

Another advantage to C# is control. There is an implied communication between a *base* class designer, and developers building subclasses. By defining a method *virtual*, the designer says, "If you feel the need to override my original method, go ahead." If he declares a *base* method as *abstract* [5] then he is saying, "If you decide to create a subclass that you want to instantiate, then you *must* implement this method, but I will provide you with no default behavior because it is unclear what I should do." This control has some practical advantages, perhaps controversial and therefore rarely discussed. Sounds like fun to me, so here goes: in most companies, there are usually at least two tiers of developers—those who build the service code, and those who build the client code. The former group (rightly or wrongly) is often perceived as being more experienced, so these developers would generally take over class design work. If this perception is correct, then the interface architecture on these base classes should therefore be more solid. C#'s idea of forcing *virtual* definitions at compile-time will give more control to the *base* class designers and constrain the development of future client development code, which may be seen as desirable.

So, back to the original question: did Java get it right, or did C# get it right? Maybe a good mature and real-world example is necessary to see some actual advantages and disadvantages of each approach.

You are the class designer, and you have been commissioned to create an *abstract* base class named `Bear` in both Java and C#, and implement a method called `HasHair` which simply returns *true*, and accepts the default *virtual* or non-virtual behavior, respectively, defined by the language. You define a `Bear` subclass called `Teddy`, create a `Teddy` instance named `FuzzyWuzzy`, and assign him to a `Bear` reference. While Fuzzy's running, you chase him down and ask him if he has hair, and he pants "yes." This seems to be right and wrong simultaneously, because while FuzzyWuzzy was a bear, FuzzyWuzzy has no hair. So in both languages, you create a `HairlessTeddy` class which subclasses `Teddy`, then override the default `HasHair` method in this subclass and return *false*. `FuzzyWuzzy`, who is still a `Bear` (and presumably always will be) now holds a reference to a newly created `HairlessTeddy` instance, and when he starts running again (he stopped to either take a breather or to have "lunch," and you'd better hope it's the former—he may be cute, but he's still a bear, and all animals get hungry) you holler, "Fuzzy, do you have hair?"—you've given up chasing him by now since, even though he's a Teddy Bear, he can still run 40 miles per hour. In Java, he now says "no," which is right. But in C# he now says "yes," which is wrong.

What's wrong? Is FuzzyWuzzy schizoid? Or has he lost his senses because he's just a little tired? Maybe both. But the real issue resides in C#: while you can create a method in a subclass with the same signature as a non-virtual base, this new[6] method will not be called unless it can be determined at compile-time that the type held is actually an instance of the subclass. And in this case using either language, it is unknown that `FuzzyWuzzy`, who was and is a `Bear`, is and was a `HairlessTeddy` until he's running. This is not a bug in C#; it's "as designed" (remember the rules above?). The class designer guessed wrong when he or she designed `Bear` (remember the disadvantages above?). He or she should have made `HasHair` a virtual method returning *true* by default because most bears do have hair, which allows but does not require subclasses to override this behavior. But how many bears have no hair? Only one that I can think of. It seems reasonable to assume that if it is a bear, that it does have hair. Who would have thought that one bear who became follicly-challenged by taking an unfortunate spin in the washing machine would refuse to call the Hair Club for Bears? So cut the class designer some slack. I'm sure that you, I mean, he or she, will thank you. (Special thanks to Rudyard Kipling on Fuzzy Wuzzy.)

So, did C# or Java get it right? No, I don't wanna. It wouldn't be fair to C# by not showing at least one example where Java can fail, so there. Here is a sample pitfall. Say that you have a class called `SmartSortedList`, which holds a list of objects. You want the client to be able to add and remove elements from the list, but you don't want to sort every time the list is modified, but rather only when the list is not guaranteed to be in sorted order *and* when the caller explicitly wishes a sort to be done. So, a private Boolean `m_bDirty` field is created with no accessors, and this field is set internally to *false* whenever the class can guarantee that its internal list is in a sorted state. When the `add` or `remove` methods are called, if the element can be added or removed, then the item is put on the back of the list or removed from it, and the field is set to *true* else it returns immediately. When the `sort` method is called and the field is *false*, then the function returns, else it sorts the list then sets the field to *false* and returns. So, the `add` method is defined and implemented in the base class, but the Java class designer forgets to make this method *final*. Some developer with incredible typing skills comes along and creates a subclass of `SmartSortedList` called `NotSoSmartMayOrMayNotBeSortedList`. Reading the helpful pop-up comments of the class designer in his favorite IDE, Visual Studio .NET, he realizes that the designer calls `pushback(Object)` in the base `add` method and the developer thinks that this is "stupid"—it should call `pushfront(Object)` instead—so the developer grumbles and overrides the add method in his derived class by just calling the *protected* `pushfront` method in the base class. (Doh! The developer should also set the dirty bit to *true*, but can't; remember the protection above?). Proud of himself, he yells, "Woo hoo!" then creates an instance of his list, adds some elements to it, and compiles his code. Unfortunately, he sorts his list just before an overriding `add` call is made, and there may or may not be a problem. He now calls `sort`, which returns immediately without sorting the list because the field is *false*. He now iterates the list, which may or may not be sorted. Good name for the new subclass. Bad night at the nuclear power plant. Hopefully it isn't disastrous for this homey. The default behavior of Java coupled with the mistake by the class designer sets the trap in this scenario, which wouldn't have happened in this case with the default C# behavior.

So, one more time, back to the original question: did Java get it right, or did C# get it right? Final answer: this is more of a philosophical argument than a logical one. The examples above can be avoided if the developers make good decisions, but they do show the problems that can arise with the default behavior of both languages. C# can simulate Java—always make every method *abstract* or *virtual* in base classes. And Java can simulate C#—define any base method *final* unless it may be overridden, or *abstract* if a subclass must implement it. Would a developer use either of these simulations? Probably not, because they each defeat the original intent and strength of the language. But they demonstrate that C# and Java are actually equivalent in this area. We've all heard the overused statement, "Building software is a set of tradeoffs which must be negotiated. There isn't a right or wrong answer." This usually just sounds like a cop-out used when someone doesn't want to make a decision, or perhaps wants to come across as being uncontroversial. In this case, "there isn't a right or wrong answer" may actually apply.

Synchronized

This keyword is context-sensitive in Java. When used as a left unary operator, it locks the instance or class mutex on an expression (generally some object) at the beginning of the block, then releases at the end of the block. In C#, the *lock* keyword supplies the same functionality.

Java also uses this as a class method modifier. If a synchronized method operates on a class instance then the instance mutex is locked and released at the beginning and end of the method call, respectively and implicitly. If used on a *static* method, it handles the class singleton lock in the same manner. If used as a class modifier, then it implicitly treats every class method as *synchronized*—a nice little "shorthand" technique.

Naturally, there are some advantages and disadvantages of this language feature. The upside: it is very simple. Either declare the class as *synchronized*, and all of the methods are synchronized; or add this modifier only to methods which read and/or write private data that need to be protected in a multi-threaded environment. In most scenarios, it is easier to avoid deadlock internally in a *synchronized* class, because only one lock is used. Of course, if the class's *private* data are also internally synchronized then deadlock is possible in the usual situations.

The downside of *synchronized* as a method or class modifier: it is somewhat simplistic and therefore encourages the developer to write code that may not be as efficient as possible. For example, many classes have more than one *private* data member. In Java, if you use *synchronized* methods or classes, you are actually locking the *this* item instead of the *that* [7] item, where the *that* item is the *private* data itself. Why should you lock this entire class instance in case you are only reading or writing one of its variables? I don't get it—you locked the wrong item! Other member variables that are not being used are now locked too, because only one thread can own the instance lock at any one time. In contrast, if the operator *lock* or *synchronized* is used inside these method calls on the private data themselves, then seemingly multiple unrelated threads can run in separate contexts and access different class variables simultaneously and eventually tie-up a surprisingly well-behaved and cohesive program. Sounds like a familiar idea, but since I'm a slow-thinker, I just can't remember.

Once again, C# can simulate Java and vice versa. Using C#, simply create a `lock(this)` statement in each method and only access data inside this block. In Java, use the *synchronize* operator on *private* data members inside of methods instead of using the same keyword as a class or method modifier. Just remember: no matter what language you're using, delay securing and manipulating your *private* members as long as you can. But once you do have them, always make sure you release as soon as you're finished. If you follow these simple rules as an application or service developer, you may feel like a new man (or woman) and sigh with satisfaction, "I am the Webmaster of my domain."

Keyword Wrap-up

C# and Java share many common keywords. C# has reserved many more, providing quite a range of additional built-in features over Java. But as stated before, more isn't always better. Developers must use this support for any noticeable difference in development ease.

It is also clear that, if you can do it in C#, you can do it in Java. So these features don't really make C# a more powerful language, but they do seem to make programming more elegant (contemplate *operator* overloading) and easier (think *foreach*), in general.

"Interesting" Built-in Class Support

There is no way to discuss all of the built-in class support that exists in C# and Java because the libraries available to the developer are huge for both. However, there are some built-in support in both Java and C# that are interesting, particularly for comparison.

How were these chosen? Well, naturally I had to have some knowledge of the classes and have used them. But another factor came into account: since this document is supposed to discuss the main language differences, what classes and libraries most help support your application in a general way? For example, while support exists in both Java and C# for accessing a database and are very useful, it could be argued that this is "less" important than say, Reflection, because the latter can be used in almost any application, while the former should only be used in those applications that access a database. It would also seem likely that the database libraries would use the Reflection libraries and not vice-versa, so this is how the distinctions were made.

Strings

Many programming languages do not have built-in strings. C++, for example, forces developers to either build their own string class or include those defined in the STL. These solutions can be less than satisfying, simply because of the lack of compatibility between varying string types. Worse: quite a bit of C++ code is littered with char*s,

which I won't even get into.

Both Java and C# have predefined String classes: java.lang.String and *System.String*, respectively. It's about time. Both are immutable, which means that when a `String` instance is created it cannot be changed. Both cannot be extended. And both hold characters in Unicode. Right on.

Also, both classes have quite a few member functions for comparison, trimming, concatenation, and so on, which makes your job easier. And these `String` classes are both used exclusively in both languages in the built-in APIs, which makes conversion unnecessary.

Threading and synchronization

In Java, any class that wishes to run as a thread must implement the interface java.lang.Runnable, which defines only one method:

public void run();

A convenience class exists called java.lang.Thread, which extends `Object` and implements `Runnable`. A developer is encouraged (but not required) to create a class that extends `Thread` and implements the `run` method. This `run` method usually just loops and does something interesting during each iteration, but in reality it may do nothing at all—you decide. Create an instance of this new class and call its start method, where `start` does some bookkeeping and then calls the actual `run` method. The reason for the `Runnable` interface: a class may exist that already subclasses another that also needs to behave as a thread. Since multiple inheritance does not exist, the "workaround" is to have this class implement `Runnable` so that it can behave as a runnable object.

This all works pretty well, and is generally very easy to implement. Naturally, there are synchronization issues that must be dealt with in case data is shared between multiple threads, but the *synchronized* keyword above helps with this. Also, Grand notes that each object has several base methods that help: wait, multiple versions, which puts a thread to sleep; notify, which wakes a single thread waiting on an object's monitor; and notifyAll, which wakes up all threads currently waiting on the monitor. These methods can be used for many synchronization techniques, including the "optimistic single-threaded execution strategy," which he describes as follows: "If a piece of code discovers that conditions are not right to proceed, the code releases the lock it has on the object that enforces single-threaded execution and then waits. When another piece of code changes things in such a way that might allow the first piece of code to proceed, it notifies the first piece of code that it should try to regain the lock and proceed." His simple example (p. 209), slightly modified, is below.

```
import java.util.*;

public class Queue extends Vector
{
   synchronized public void put(Object obj)
   {
      addElement(obj);
      this.notify();
   }

   synchronized public Object get() throws EmptyQueueException
   {
      while (size() == 0)
```

```
        {
           this.wait();
        }
        Object obj = elementAt(0);
        removeElementAt(0);
        return obj;
     }
  }
```

So, the code that accesses the `Queue` on a `put` simply must gain the instance lock (which it does implicitly because the method is *synchronized*), add the element, wake up the first thread waiting on the lock, then return. The `get` method is a little trickier: it must gain the instance lock, loop and sleep until the `Queue` is no longer empty, remove the first element from the java.util.Vector base class and return it. The "putter" does not have to call `notify` or `notifyAll` in this sample because Grand is assuming that only one thread is putting data on the `Queue` and one thread is removing data from the `Queue`, and the former never sleeps. Must be a tough gig.

Naturally, there are many other possible scenarios, but this example shows many of the basics of threading and synchronization in Java.

C# is somewhat different. While there is a *System.Threading.Thread* class in C#, it is *sealed* which means that it cannot be subclassed. A `Thread` constructor takes a *ThreadStart* delegate, created by the developer, which is a function that is called after the *Start* method is called on the `Thread` instance. This *delegate* function is analogous to the Java's `run` method, discussed above.

While simple synchronization can be done in C# using the `lock(object)` statement, more complex operations, like the Java example above, are performed by using a separate *System.Threading.Monitor* class. All of the methods in this *sealed* `Monitor` class are *static*, which can be shown to work nicely by comparing this with MFC. In MFC, it is common for a class to own a corresponding *CCriticalSection* member for each synchronized member variable held in a class. Before the synchronized member is inspected or modified, the related CCriticalSection's `Lock` method is called, keeping out any other well-behaved thread temporarily on the data. When the code block is done with the synchronized member variable, the CCriticalSection's `Unlock` method is called. While this is a reasonable solution that works, there is a drawback: for each piece of data shared by different threads, the corresponding `CCriticalSection` object must also be passed. In some situations, the use of multiple inheritance, or creating a class wrapper that holds both objects, may be a nice workaround, but may not always be possible. And it will always require more development work at the least. I'm not "knocking" MFC here; this class was created because there is no idea of a single lock on all C++ class instances, because classes do not subclass some abstract base class like object, discussed above. So MFC had little choice in this design decision. In contrast, with the C# scheme, since the `Monitor` class methods are *static* and accept strictly the object to synchronize as a parameter, objects can fly around "solo" and be synchronized easily through this `Monitor` class, where the `Monitor` acts like a well-trained air traffic controller by helping objects avoid nasty midair collisions. To access the data in the simplest manner, call the *Monitor.Enter(object)* method. When done, call the *Monitor.Exit(object)* method. The other methods that are available are *Pulse* and *PulseAll*, similar to Java's `notify` and `notifyAll` respectively; *tryEnter*, which is a nice addition because it allows you to test a lock with an additional timeout, including zero; and *Wait*, which is similar to Java's version but has the added feature of allowing the timeout mechanism, again.

One caveat exists while using `Monitor`: in between the `Enter` and the `Exit` calls, the developer must be careful that any exceptions thrown must be caught, because if the `Enter` call is made but the `Exit` call is not made explicitly, then any thread attempting to access this data again will block permanently in some situations. In contrast, if the `lock(Object)` statement is used, and an exception is thrown inside this statement, the object will be unlocked implicitly by the compiler just before the *lock* statement is exited. But this is pretty standard stuff; the developer has to be responsible for handling some logic.

Which approach is better? That is a tricky call. Java is maybe simpler for the developer: just create a `Thread` subclass, implement a `run` method, and `start` the thread. C# requires a little more work perhaps, but seems to be a little more powerful when it comes to synchronization support. And C# may be a little more flexible in the same way that its events are more flexible: the name of the `ThreadStart` *delegate* can be anything, which avoids name collisions, and does not require another class to be created to define a thread. In contrast, the Java developer must create a specific class with a `run` method with the exact signature defined by the `Runnable` interface. Overall, the "which approach is better" question is probably moot. It's most likely a tie, because they are very similar. What's really important: native thread and synchronization support in both of these languages, which allows for much greater code portability and ease. It's a win-win scenario.

Reflection

Both Java and C# have support for reflection, which allow a developer to do things like:

- Create an instance of some class at runtime by only knowing the fully qualified class name as a string
- Perform a dynamic method call on some object at runtime by either dynamically searching the object for the methods that it supports, or using the method name and parameter types to find and invoke the method
- Set property values at run-time on an object by dynamically querying this object for a property by name, then setting that property by passing a generic object to it

For Windows C++ developers, this is somewhat similar to functionality supported in ATL. Grimes (p. 206) notes that if you define an ATL interface by deriving from IDispatch, then dynamic calls in COM may be performed, for you COM developers.

What good is this stuff, anyway? Why not just create an instance of an object and make method calls on it directly? That has to be faster. In fact, developers often complain about speed when using reflection, saying that reflection is too slow, which sometimes translates into an excuse for not using it. But it does come in handy in some situations. Say that you have some configuration file, perhaps an XML document, which has some schema that describes some data, and the properties and methods that should be called on that data, to build any object dynamically in a recursive manner. So, it could look something like:

```
<?xml version="1.0" encoding="utf-8" ?>
<object>
   <name>MyProject.MyClass</name>
   <properties>
      <property>
         <name>ClassProperty</name>
         <object>
            <name>MyProject.PropertyClass</name>
            <properties>. . .</properties>
            <functions>
               <function>
                  <name>MyFunction</name>
                  <params>
                     <object>. . .</object>
                  </params>
               </function>
```

```
            </functions>
         </object>
      </property>
      <property>
         . . .
      </property>
   </properties>
</object>
```

In C#, you could write code that would use the *System.Xml.XMLDataDocument* class perhaps, that loads and parses an XML document, then allows you to walk the DOM at runtime. Every time that an `<object>` node is seen, then a new object should be created using its fully qualified name. Every time a `<property>` node is seen, then its parent node's property should be set to some newly created object that will be shortly defined. And every time that a `<function>` node is seen, then the parent object created should have the function, by name, called on it using the parameters that are later defined. This would allow you to create any object type that you want, and set properties and call functions for initialization, in a very recursive manner. The code that does this will be surprisingly small because it takes advantage of recursion and the generic nature of reflection.

Naturally, this can be overdone. The compiler will not give you very much help when you use reflection in either language, because it has very little knowledge of what you are intending to do, what object types you will hold, and what functions you will call, until run time—almost everything is handled as the most abstract class, object. And, sure, it can be slow; much slower that making direct calls on a well-known interface. But if you use reflection only at the time that data is created and initialized, then take this data and make direct calls afterwards perhaps by casting some object returned to some known interface or your own base class, then very dynamic and powerful code can be written that allows you to modify program behavior without recompiling—just modify the XML document and refresh will work in this scenario—while still enjoying an application that runs at a reasonable speed.

While C# and Java are very similar, there are some differences in packages and libraries used. As a C# example, you might use the following classes and steps to create a new class instance and call some function on this new object (we are assuming that no Exception is thrown during these steps for simplicity, which you should not do in your code unless you can absolutely guarantee correct behavior—and by "absolutely" I mean agree to resign your cushy development position if it fails—at least this is what a boss once told me). It should be noted that there are many ways to do this, but this one will do:

1. Use one of the *System.Reflection.Assembly* class's *static Load* methods to return the correct *Assembly* which defines your class.
2. Call the returned *Assembly* instance's *CreateInstance* function which returns an object reference to your newly created class instance, as long as the class to instantiate has a constructor with an empty parameter list.
3. With the returned new object, call its *GetType* method to return the run-time *System.Type* of this object.
4. With this `Type` object (a `Type` is an object!), call its *GetMethod* function by passing the name of the method as a string (and maybe the types of the parameters if the method is overloaded) which returns a *System.Reflection.MethodInfo* object.
5. Call this MethodInfo's *Invoke* function, by passing the new object returned from the *Assembly.CreateInstance* method above, and an array of object parameters on this object's function if necessary, and accept the object returned which represents the data returned from the dynamic invocation.

Pretty simple? It's actually not as bad as it sounds, particularly when you write code that handles creating objects and method calls in a generic way. Returning to our xml sample above, it would be possible to create one function that hides these details from us, so we only have to "jump through some hoops" once by writing code that creates some object. Here is an equivalent set of steps using Java:

1. Use the java.lang.Class's *static* method forName(String) method, passing the fully qualified name of the class to create, and receiving a the *Class* object with name *className*.
2. With this `Class` object, call its newInstance method, which returns a newly created object of type `className`, as long as the type to create has a constructor with an empty parameter list.
3. With the Class above, call its getMethod(string, Class[]) function, by passing the name of the method and a list of parameter types to pass to the method and receive a java.lang.reflect.Method object.
4. With this *Method* object, call its invoke method by passing the newly created *Class* instance above, and an array of actual object parameters, and receive the return object from the call.

I would recommend that you take a day or so to "play" with reflection in either or both languages, just to see what you can do. It can be fun, and it can be powerful. Down the road, I guarantee that you will run into a situation where this feature can really make your programs more dynamic, and you will be glad that you know the basics so that you will be influenced to take advantage of this powerful language feature. As Socrates once said: "How can you know what you do not know?" I think that he was referring to reflection. One day now may save you weeks-or-more development time later if you are unfamiliar with this support and don't use it in your design early.

While I have not experimented with the IDispatch *functionality in COM above from a client's perspective, I have used reflection in both C# and Java, and they are very similar. From my experience, C# is a little trickier to "get started," because the* System.Reflection.Assembly *class is used first to load an assembly which defines and implements classes of interest. With the fully qualified name only of some class to load, you may have to write some C# parsing code to search first for the Assembly part, get a reference to this Assembly, then use the Assembly with the full name to create an object. In Java, simply use the fully qualified name to create an instance of a class using class Class (nice name for a class?). Naturally, there are tradeoffs, once again. The Java code may be easier to create an object, but it also may be the case that the C# and CLR infrastructure are easier from an administrative viewpoint over the additional classpath information which must be configured in your system using Java, regardless of operating system. But once you have this new object, both languages seem to be similar when it comes to making dynamic function calls and setting properties, for example (of course, Java does not support properties directly, however).*

Hits and Misses

Both C# and Java have taken a different approach to programming than many languages that have come before them. Some of these are "hits," and some of these are "misses" or near misses. And some things could still be improved. But the majority of changes are very good.

What Have Both C# and Java "Fixed?"

There are some things that both C# and Java have fixed. Some of them are listed below.

Boolean expressions

C++, C#, Java, and other programming languages, support *if* statements of the form

if (expression) statement1 [else statement2]

While *expression* in languages like C++ can be nearly any expression, C# and Java require it to at least be castable to a Boolean type. So, the following *for* statement is legal in C++

```
int i;

if (i = 3)
{

}
```

because any expression that returns any non-zero value is considered to be true in C++. This statement would not compile in either C# or Java, because expression `(i = 3)` is not a Boolean expression; "3" cannot be implicitly converted to a Boolean. Rather, it is an assignment expression that simply returns the value "3."

You may ask, "Why is this in the 'fixed' section? This seems to actually make things more difficult in C# and Java?" Well, if everyone could decide which values that "true" and "false" should map to, then you could reasonably argue that the C++ method is better. But even some C++ extensions have their own rules. For example, when defining Boolean types in ATL, VARIANT_TRUE must be -1 and VARIANT_FALSE must be 0. The Visual Studio 6 online documentation states: "To pass a VARIANT_BOOL to a Visual Basic dll, you must use the Java short type and use -1 for VARIANT_TRUE, and 0 for VARIANT_FALSE." So, if you load a dll in Java for a native call on a Windows operating system, a Java *true* value must *first* be mapped to -1 (minus one). Sounds pretty confusing. C# and Java therefore say: "*true* must be *true*, and *false* must be *false*." Sounds pretty ingenious. But this obvious change eliminates most programming error and ambiguities with their newly defined Boolean expressions.

Arrays

Developers love arrays. Always have, always will. But arrays can be the cause of many headaches, particularly in languages such as C++.

Both C# and Java treat arrays as first-class objects. After creating an array, you may ask it its name, rank and serial number. Or at least its *Length*. And if you attempt to access the fifth element on an array of length five in these languages (potentially OK in Pascal, big trouble in C++) then the array will *throw* an Exception telling you that you are out of bounds.

Naturally, in languages like C++, a developer can simulate this functionality through the use of templates. Lippman (pp. 480-4) has a decent specification and implementation of a range-checking array template that will do this for you. But both C# and Java already have this functionality built-in for every new array that is defined. While bounds checking is still optional in C++—just don't use a wrapper—it is mandatory in both C# and Java.

Of course, people will complain about C# and Java, saying that it is slower than C++, particularly when using arrays. The speed tradeoff is well worth it, and in reality, is fairly minor anyway. Let C# and Java take the wheel, and ease off the gas pedal just a bit. Sometimes speed does kill, and both will pop out like a life-saving airbag when you most need it.

What Has C# "Fixed?"

There are disadvantages in being very young: You don't get much respect, and you have to go to school. But there are some advantages: You can learn from other's mistakes if you pay attention, and you don't have to pay taxes. At least I know that C# has gone back to school and it has been paying attention. Kinda like Rodney Dangerfield, except that it may get some respect someday.

For statement

For loops in almost any language are looping constructs which allow a developer to perform an internal code block any number of times desired. C#'s *for* loop takes the form:

for ([initializers]; [expression]; [iterators]) statement

which is similar to C++'s version. In C++, any variable defined in the initializers statement are available after exiting the *for* loop. This is problematic, because in some complex scenarios, it is not so obvious what the value of these variables should be after the loop terminates. And not only that, it just "feels" wrong because *initializers* at least visually appears to be part of a scope that no longer should be valid.

C# has "fixed" this by hiding these variables after the *for* loop exits. This might make you mad because you now may have to create another variable before the *for* loop and side-effect this variable during each loop iteration if this information is necessary after loop termination. This might make you glad because your C# code may actually work correctly with this change. Either way, it's probably the right thing to do.

I wish that C# would have made one additional change. I believe that a for loop's statement should actually be a block, requiring the use of { and } to wrap a statement list. Why? Even though curly brackets are currently optional when the for loop is designed to execute a single statement, I always use them in any looping statement, for a couple of reasons.

First, it is a well-known problem that if a developer comes along later and decides to add additional statements to this list and does not add the brackets, then only the first statement will execute. For example:

```
int j = 3;

int k = 5;

for (int i = 0; i < 5; i++)

j++;

k--;
```

In the above for loop, it is obvious to you and me that the developer's intent is to increment `j` *and decrement* `k` *during each loop because of the indentation level used. But it is not so obvious to the compiler; the* `k--` *code will not execute until the for loop terminates, meaning that code above will only be correct if all the planets align in the southern sky. The second reason is simply readability. It is easier to determine the programmer's intent when brackets are used in code.*

It seems that C# was modeled after C++ syntax, so this might have been the reason that C# did not force this change. I'm not sure. Just because the language doesn't enforce it doesn't mean that you shouldn't do it. I would recommend using the brackets, but I still wish that C#, and other languages, enforced them.

Switch statement

The *switch* statement is a control statement available in many languages whose logic is similar to an ITE statement, except that a *switch* statement generally deals with discrete values, while the ITE can examine more complex expressions. The nice thing about an ITE: only one block should be executed. The bad thing about the *switch*: multiple blocks of code can be executed, even if the developer's intent was to only execute one. One language where this can happen is C++, which allows "fall through" on *case* clauses. This so-called "feature" was added because the language wanted to give developers the ability to use one code clause to handle more than one input

value in case the exact same action should be taken in a set of possible values. C++ requires the use of some jump-statement at the end of each clause only if the developer does not want "fall through" to happen. Naturally, developers often forget to jump, and when they do, the most likely scenario is that their code will not work as they intended. It's funny: Our *Jackass* friends above get hurt when they jump, while our C++ buddies get hurt when they don't. Go figure. . . .

Anyway, one interesting language is Pascal and its use of a *case* statement. The *case* is very similar to C++'s *switch*, but the former has no "fall through" mechanism. At most only one clause may execute in the *case* statement, which is not a problem as it also allows the developer to create a comma-delimited list of constant values which map to its clause. For example:

```
//global function
integer SeeminglyUselessFunction(integer i)
begin
   case (i)
   begin
      1, 2:  //do something if i is either 1 or 2
      3:  return i;
   end
end

//client code
integer i := 3;
integer j := SeeminglyUselessFunction(i);  //j either gets i or the program crashes!
```

Forgive me if my code won't compile—it has been a long time since I wrote a line of Pascal. And even if it will, I'm not saying that the syntax above is beautiful. Headington (p. A-38) has even observed that Pascal does have at least one problem with its *case* statement: if the expression value to the statement is not found defining any clause then the application behavior is undefined, as no *default* clause is allowed. So, if `i` were actually set to 4, then I can't guarantee well-behaved program behavior, and neither can `i`. This means that *you* must usually "guard" a *case* statement by wrapping it inside an *if* statement, where the if's boolean expression verifies that the input value is in a predetermined set or range where the value is guaranteed to exist in some clause, which can be nasty. No, Pascal is not perfect, and neither is its *case* statement. But the comma-delimited list of discrete values is a great idea, and it's somewhat surprising that no one seems to want to "borrow" this idea (except for me, below, in my "new" programming language). And not allowing "fall-through" eliminates much error.

C# has taken somewhat of a different approach. For one thing, it not only accepts integral values as input but it also allows string constants. For another, C# does not allow "fall through" in its *switch* statement, like Pascal, which eliminates errors. How does it do this? Each clause is required to end with some jump-statement. All in all, C#'s version of the *switch* statement is an improvement over most if not all languages that came before it. But what I would really like to see is some language that uses a combination of C#'s *switch* and Pascal's *case*. The following rules could be defined:

- No "fall through" allowed, like both languages.
- Jump statement not required at the end of each clause, like Pascal.
- Each clause can map to a comma-delimited list of constant values, like Pascal, and maybe even a range of values!

- *default* clause allowed, like C#.
- *const* string values accepted as input, like C#.
- (Maybe) allow non-const test expressions for clause execution (no one has allowed this yet so I'll admit that this might be a bad idea). One potential issue: two or more complex expressions could be written by the developer to define a clause which could return *true*, which would cause some ambiguity over which clause to execute. But it would be interesting to look into, at least.
- Replace *begin* and *end* with *{* and *}* already.

So, the sample above in the new language K++ could now look like:

```
class MyClass
{
   static int SeeminglyUselessFunction(int i)
   {
      switch (i)
      {
         case 1, 2 :  //no-op - leave statement
         case 3:  return i;
         default:  return i;
      }
      return I;
   }
}
```

Method accessibility keyword modifiers and meanings

Both C# and Java allow accessibility modifiers for methods. *public*, *protected* and *private* are allowed by both.

Java is inconsistent and therefore unintuitive with the meanings of these keywords. For example, *public* means that a method is accessible to anyone outside the class, whether the method is static or not. *private* means that the method is only accessible from within the class. These make sense. But *protected* means that *any* code inside the enclosing package or subclass can access the method, which seems to be mixing metaphors. For example:

```
public class MyClass
{
   public MyClass()
   {
   }

   protected void Test()
```

```
    {

    }
}

//code outside of the MyClass class in the same package
MyClass mc = new MyClass();
//from the global package level
mc.Test();  //this is allowed in Java!  It really shouldn't be.  Neither C# nor C++ allow this.
```

C# has not only "fixed" this problem, by defining *protected* as meaning that only the class or its subclasses may access the method, which is more consistent with the *private* and *public* modifiers and models the behavior of C++. But it also has added a couple more choices: *internal*, whose meaning is very similar to Java's *protected* keyword; and *protected internal*, which is once again similar to Java's *protected* plus access by any subclass.

Java's use of the method modifier protected causes much confusion to developers, particularly to those that know C++. How do I know this? While I was coding in Java professionally, I used the protected keyword for quite awhile without knowing the meaning! I thought that a protected method could only be called by the class and any derived class! During some testing I noticed some strange behavior, and I initially believed that the compiler had a bug. I eventually had to return to my code and change protected methods to private, and in some cases, create accessor functions in base classes so that subclasses could access this data. Not nice, and not a good idea.

Naturally, I should have done a better job of reading documentation. But it didn't even occur to me that the meaning of this keyword should or could have a different meaning, so I didn't even consider it.

try-finally statement issues

Gruntz notes on his Web page an issue with Java's try-finally statement, where this problem does not exist in C#. He notes that if an exception is thrown, or a *return*, *break* or *continue* are used in a *try* block, and a "*return* statement is executed in the *finally* block," then undesired and unexpected program behavior may occur.

He also notes that this issue does not exist in C# as this language "does not allow (the developer) to leave the control flow of a finally block with a return, break, continue or goto statement."

Constructors

Two other issues in Java are noted by Gruntz on his Web page, both dealing with Constructors.

The first is a "phantom" constructor. Java allows a method to have the same name as the enclosing class, while C# does not. The second issues deals with calling non-final methods from constructors in Java, which causes problems.

C# has fixed both of these issues by adding certain restrictions, which he argues are reasonable. He also discusses other issues, which are quite interesting to read.

What Has C# "Broken"

Scope issues

It appears as if C# might have a bug with variable declarations in some scope situations. For example, the following C++ code will compile, but will not compile in C#:

```
for (int i = 0; i < 3; i++)
{
   int j = 3;
}

int j = 5;
```

The C# compiler complains that `j` has already been defined in the second `j` assignment statement. It should be clear that the *for* loop's block is in a different scope as this second `j`, and that once the *for* loop is exited the first `j` should no longer be accessible. Therefore, this code should be legal. I hope that the first `j` is not accessible once the for loop terminates!

Bug? As designed? My guess and hope is the first, because I can't determine the motivation for making this illegal.

Enum issues

While it is great that C# allows enumeration types, it is still possible to set an enumeration variable to a value not sanctioned by the enum's enumerator-list. For example:

```
enum Colors{red,green,blue};
```

defines a list of three possible color elements. If the following code is written, compiled and run:

```
Colors c = (Colors)2;
```

then `c` has the value `blue`, which appears correct, as all elements by default in an enumeration-list are zero-relative by default. But if the following code is used:

```
Colors c = (Colors)4;
```

then `c` will have the value of `4`. It seems as if the compiler should be able to flag this particular line of code as an error at compile-time, because "4" is a constant, and the min and max elements in `Colors` are 0 and 2, respectively. Even if a variable were used where its value could not be known until runtime, it seems as if an exception should be thrown in this out-of-bounds scenario to guarantee that an *enum* type is truly "type-safe."

It's great that C# allows enum *types. Java does not, which I still believe was a mistake. And it's great that an enumeration variable may be set to a numerical value using a cast, for many reasons. But it should be possible to guarantee that* enum *variables are not set to illegal values, either at run time or compile time. In the above example, a* `Colors` *variable holding the value of '4' seems to have no meaning. What does '4' mean? What do you do in the scenario where some calculation must be performed as a function of a* `Color` *whose value is '4'? It's not so clear.*

Why did C# allow this? Was it for interoperability issues? I'm not sure yet. Just be careful.

What Would Have Been "Nice" Additions to Both C# and Java

Const method and formal parameter keyword modifier

It would have been very nice if both languages would allow the use of *const* as a method and formal parameter reference modifier, with the same meaning as C++. Since all objects in both C# and Java are references, if a parameter is passed by a caller and side-effected in a method by the callee, then the former will see any changes made to his data after the call is complete. Sure, many C++ developers have complained about the use of *const* in C++, arguing that the "constness" of a parameter can be cast away anyway, making this functionality useless. However, it seems as if a simple rule could be added to C++ stating that "a *const* reference may only be cast as another *const* reference," which could be tested at compile-time and eliminate this problem at runtime. If this is true, then C# and Java could have included this functionality with this modified rule as well.

In addition, an implied guarantee is only part of the contract between the caller and the callee anyway on a *const* function or parameter. Adding this *const* keyword gives a reminder to the caller and callee that the data is not supposed to be modified, which creates more self-documenting code. In particular, a developer following good programming habits will be reminded by the compiler with an error message if he is trying to modify *const* data. If he is trying to circumvent the rules, then he will be the one who suffers anyway, even if the compiler cannot catch the illegal operation.

Readability is an important part of coding. This is a reminder of the debate over functions that return multiple values by side-effecting input parameters, which in some scenarios is reasonable. Some developers prefer non-const references, while others prefer pointers. It can be argued that the latter is "better." Why? Headington (p. 303) has two versions of a common swap function that will help me argue the point:

```
void Swap(float* px, float* py)
{
   float temp = *px;
   *px = *py;
   *py = temp;
}

void Swap(float& x, float& y)
{
   float temp = x;
```

```
    x = y;
    y = temp;
}
```

The client code to use these functions could look like:

```
float a = 1.5;
float b = 2.0;

Swap(&a,&b);  //calling the pointer version
Swap(a,b);  //calling the reference version
```

Looking at the client code, the second `Swap` appears cleaner. But if you write client code using the pointer function instead, then return to inspect your code much later, you are reminded that your parameters may be modified during this pointer `Swap` function call because of the additional "&" character on the client side, which is important, in a good way. However, the `Swap(a,b)` call gives no reminder to this effect, which may also be important, in a bad way. While both functions may be logically equivalent, it can be argued that the pointer method is "better" because of the extra communication that exists between the client and the server code, and increased readability.

After playing with managed C++ code, which is actually pretty interesting but beyond the scope of this paper, I realized that this new .NET functionality also disallows the use of const *as a method modifier. I received the compile-time error message: "'const' and 'volatile' qualifiers on member functions of managed types are not supported." And while it is legal to use the const modifier on a reference parameter, the constness must be cast away inside the method to make* any *method call on the parameter! Presumably, this is because the compiler cannot guarantee that any method called on the object address will not modify the object or its data since the* const *modifier is disallowed on instance methods, noted above. This begs the question: is it a waste of time to use the* const *modifier on reference parameters while using managed C++ code? The answer: not completely, as long as the class designer and implementer takes the above restrictions into account by defining, through comments, if an instance method should be in fact a* const *member function. Then if a* const *parameter address is received by a method, the developer can "safely" cast away the constness but call only simulated* const *methods on the object. This way, if managed C++ code is eventually ported back to unmanaged code, then the comments could be removed and the classes will work as designed, minus removing that nasty cast, of course.*

I realize that this advice immediately opposes my advice above: don't cast away the constness of a formal parameter address. But in my opinion, this does not break the "spirit" of the original advice, because the class builder still guarantees that the client data will not be modified, although this guarantee is now made by human inspection rather than compiler logic. And it is a workaround to a limitation that allows greater ease of future C++ portability, if necessary.

All this leads me to believe that there may be some language interoperability issues that forced Microsoft's hand, because a const *method must be negotiated by both caller and callee, but a* const *parameter address really only needs to be guaranteed by the latter (the C# code calling a method written in C++ couldn't care less about the const reference anyway, for example, since this can't be done in C#). If all this speculation is true, then I am assuming that Java may also have had the same problem during initial design since, even though it is trickier, Java can call functions written in other languages. Maybe managed C++ should have allowed the* const *modifier on an instance method*

with the understanding that the developer was in charge of verifying that no change is made to the internal data. Or maybe the const *keyword should have been disallowed as a formal parameter modifier. I don't know. From a portability standpoint, the former solution might have been a better choice. From a behavioral viewpoint, the way that managed C++ works now makes some sense, since it works as designed. At any rate, this seems to be somewhat inconsistent, allowing* const *parameters but disallowing* const *instance functions with managed C++.*

I would recommend reading information about Cross-Language Interoperability *in the online documentation. I know that I am going to have to now look into this some more myself. . ..*

Access modifiers for class inheritance

Languages like C++ allow the use of class access modifiers while using inheritance. For example:

```
class MyClassBase
{

}

class MyClass : public MyClassBase
{

}
```

in C++ means that `MyClass` subclasses `MyClassBase`, and any code holding an instance of `MyClass` may call any function declared *public* defined and implemented in `MyClassBase` on this instance. If we used *protected* instead, then *public* methods on `MyClassBase` would be private to the outside world when a `MyClass` instance is held. *private* is also possible, but you get the drift.

Neither Java nor C# allows the use of *public*, *protected* or *private* accessors in this manner, which is too bad. There are instances where a class should logically inherit from a base, where not all base methods should be accessible outside of either. For example, let's say that we have a class called `AddArrayList`, which extends the *System.Collections.ArrayList* class. The idea of our new class is to allow the client access to an array of elements, where this array can be added to, but no element can be removed, and therefore only allow a very small subset of functionality that the base class supports. If C# allowed accessor modifiers, we could then define our `AddArrayList` in the following manner:

```
//currently illegal C# code
public class AddArrayList : protected ArrayList //currently protected disallowed here
{
   public override int Count
   {
```

```
        get
        {
           return (base.Count);
        }
     }

     public override object this[int index]
     {
        get
        {
           return (base[index]));
        }
        set
        {
           base[index] = value;
        }
     }

     public AddArrayList()
     {

     }

     public override int Add(object value)
     {
        return (base.Add(value));
     }

  }
```

Since this is not possible in either C# or Java, we would need our `AddArrayList` class to hold a *private* data member of type `ArrayList` and create our methods that operate on this *private* data member. This sounds OK. After all, it's really no more difficult than the above solution. But it could be argued that `AddArrayList` really is a special type of an `ArrayList`, so the former should really subclass the latter.

Naturally, this can be abused: creating a `List` class for example that subclasses a *protected* stack would probably be inappropriate. But there are some situations where this makes sense, so it would have been nice if it were supported.

Conclusion

So, which language is superior? You have Java, which will run on many different operating systems without recompiling code. You have C#, which seems to have many more built-in language features, and will run on any operating system that has an installed CLR. Naturally, if you have limited development time and need an application

that can run on almost any operating system, then Java seems to be the obvious current choice. But if you know that your application will run on Windows, and you have or don't have limited development time, using a good type-safe, powerful language like C# seems to a very excellent decision. Anywhere in-between, which most software falls, will require a more difficult analysis.

I have used both languages professionally, so I know about some of the strengths and weaknesses of both. If it were my choice, and I were to create a new application that I knew would run on a Windows operating system, then I would choose C# over Java. From my experience, I know that even Java applications don't always behave the same way on different operating systems, particularly when building user interfaces. But I'm not trying to "dis" Java; it's a minor miracle that it works at all on so many platforms. Rather, I would choose C# over any language that I've used before, including C++, Smalltalk, Pascal, Lisp, and again, Java—you name it. C# is a very good pure object-oriented programming language, with lots of features. It is quite evident that the architects of C# spent a lot of quality time and quantity effort to build such a potentially quintessential language. It does have a few debatably minor snags, some of them described above, but the strengths far outweigh any of its minor weaknesses. But what I think doesn't really matter. What does matter is what language you will use to create your next application. That is up to you.

If you have read this document to this point, I thank you. It is lengthier than I originally planned, but it could in reality be much longer—there is so much left to cover. But the ironic conclusion that I must come to is this: it doesn't matter which language is better, and maybe more important, which language to use. Why, you may ask, after all your devoted reading? Because C# doesn't even care. If you read the section above about language interoperability, you realize that C# doesn't even know what languages were used in the libraries that it imports and uses. To C#, compiled J# looks just like compiled C# looks just like compiled managed C++ looks just like compiled whatever-new-language-is-supported. If C# doesn't care, why should you? And in reality, the "C# versus Java" debate is just an either-or fallacy anyway; there are of course many language choices at your disposal. Use whatever you want. Personally, I will continue to use C# because I think that it is a solid language. But the introduction of Visual Studio .NET should make comparisons moot, because more and more languages should be supported by this development tool and platform in the near future, allowing you to use whatever language you choose. And at that point, any choice you make shouldn't be a bad one.

Maybe we don't have to dream about that "fantasy world" any longer.

Special Thanks

I would like to thank Mike Hall from Microsoft for taking the time to read my initial outline, and providing some excellent suggestions. I would also like to thank Steve Makofsky for sending me emails every time he found some interesting information on the Web about C# and/or Java. Last but not least, Ginger Staffanson, for "convincing" me to write these white papers, and providing the initial editing pass.

Notes

[1] I ran some simple tests in Visual Studio 6 to see how well this IDE handles ambiguities for the developer. I created a class that subclassed two bases, where each of the bases shared a super class of its own. I created and implemented a virtual method in the super, then implemented it in both of the bases. I tried to create a class instance and it failed to compile. Then I removed the virtual method and created a private data member in the super class. Once again, I tried to create a class instance and it failed to compile. In both scenarios, the compiler wouldn't allow me to create an instance of the class because of the respective ambiguity. Replacing the virtual method in the super, I finally had to define the super class as a public virtual inheritance in each of the base's specification file, remove the virtual definition and implementation from one of the bases, and then create a class instance and assign a super pointer to it. The code compiled. Check. At run time, the instance had only one copy of the super's private data. Check two. The correct version of the virtual function was called on the super pointer. Check three. The compiler even warned me which virtual implementation would be called "via dominance." Bonus check. This IDE seems to do a good job of flagging ambiguities for the developer, and I am assuming that Visual Studio .NET may do even better. This seems to imply that, sure, multiple inheritance can cause problems, as many have argued before. But a good compiler can really help you avoid them now.

[2] To be completely fair, I have not seen *Jackass the Movie*, and probably never will. Why, you may ask? Well, if I were to call this paper *C# and Java for Dummies*, would you read it? One more obligatory item: "We are not responsible for anyone who attempts the programming stunts discussed in this paper. Leave it up to the professionals."

[3] A set of simple classes were built to derive these steps. I created a Button subclass that listened to itself for a press, then fired a new event type that I created to anyone listening for this new event. I then created a class that implemented the corresponding delegate method, and an instance of the Button and the listener and added the latter to the former's event list. Everything worked as advertised. While this is fairly straightforward, it is probably a good idea to do something simple like this at first before trying to do anything more complex.

[4] I have only a little experience with J#. I downloaded the plug-in and installed it into Visual Studio .NET. While most of the language syntax remains faithful to Java, the APIs available are the same as those available to C# and C++, because these are meant to be managed code solutions which can play nicely together. So, classes such as *ArrayList* would be used in C# and J#, while classes such as *Vector* would be used in Java.

[5] An *abstract* method in both C# and Java are used generally in base classes, which tells any subclass which wishes to be instantiated that it must provide definition and implementation code for this method. The *abstract* method definition is simply that; it has no implementation. It is very similar to any method defined in an interface.

[6] Any method with the same signature in a subclass as a base class, where the latter's method is non-virtual, should include the keyword modifier *new* in the more specific class. While this is not required in this case, it signals to any subsequent client that this attempted overriding method will not be called in case a more abstract class reference holding a more specific instance is held at runtime.

[7] *that* is not a keyword in either language. It's only being used that way for effect, not that there's anything wrong with that. . . .

Works Cited

Albahari, Ben. A Comparative Overview of C#. Genamics. 2000.

Computer Science Department, University of Massachusetts at Boston. Java Keyword Index. Boston, Mass, 1997.

Grand, Mark. *Java Language Reference*. 1st Edition. Cambridge: O'Reilly, 1997.

Grimes, Richard, et al. *Beginning ATL 3 COM Programming*. Birmingham, UK: Wrox, 1999.

Gruntz, Dominik. C# and Java: The Smart Distinctions. Aargu, Switzerland.

Headington, Mark R., and David D. Riley. *Data Abstractions and Structures Using C++*. Lexington, Massachusetts: D.C. Heath and Company, 1994.

Lippman, Stanley B. *C++ Primer*. 2nd Edition. New York, NY: Addison Wesley, 1991.

Saraswat, Vijay. Java is not type-safe. Florham Park, NJ: AT&T Research, 1997.

Schildt, Herbert. *STL Programming from the Ground Up*. San Francisco, CA: Osbourne/McGraw-Hill, 1999.

© 2017 Microsoft